Rotary (Furuta) Inverted Pendulum: Modelling and Experimental Comparison of Linear and Sliding-Mode Controllers

Piyush

April 14, 2026

Contents

0 Problem Framing, Contributions, and Reading Guide **6**
- 0.1 Problem . . . 6
- 0.2 Key contributions (summary) . . . 6
- 0.3 Reading guide . . . 6

1 System Hardware and Constraints **8**
- 1.1 Overview . . . 8
- 1.2 Hardware build-up from scratch . . . 9
- 1.3 Mechanical rig . . . 9
- 1.4 Electronics and actuation . . . 11
- 1.5 Sensing . . . 11
- 1.6 Bill of materials . . . 13
- 1.7 Host computer and data acquisition . . . 13
- 1.8 Safety limits and operating envelope . . . 13
- 1.9 Replication notes and lessons learned . . . 14

2 Notation, Units, and Recorded Signals **15**
- 2.1 Coordinates . . . 15
- 2.2 Sign conventions . . . 15
- 2.3 Wrap-safe angle differences . . . 15
- 2.4 Units policy and stepper conversion . . . 16
- 2.5 Variables and recorded signals . . . 16

3 Mathematical Modelling from First Principles **18**
- 3.1 Definition . . . 19
- 3.2 Parameter evaluation . . . 19
- 3.3 Kinematics . . . 21
  - 3.3.1 Velocity components . . . 22
  - 3.3.2 Speed squared . . . 22
- 3.4 Energies . . . 23
- 3.5 Euler–Lagrange equations . . . 24
  - 3.5.1 Equation in θ . . . 24
  - 3.5.2 Equation in α . . . 25
- 3.6 Nonlinear model . . . 25
- 3.7 Linearization about upright ($\alpha \approx 0$) . . . 26
- 3.8 Numerical specialization . . . 26
  - 3.8.1 What is used later . . . 27

4 Firmware Pipeline: Measurement and Estimation **28**
4.1 Control loop timing . . . 28
4.2 Derivative estimation . . . 28
4.3 Wrap safety and glitch rejection . . . 29

5 Firmware Pipeline: Actuation and Safety **30**
5.1 Stepper actuation pipeline . . . 30
5.2 Soft limits and failure modes . . . 31
5.3 Auto-trim and velocity leak . . . 31

6 Linear Full-State Feedback Controller **32**
6.1 Linearized plant for control design . . . 32
6.2 Full-state feedback by pole placement . . . 32
6.2.1 Desired poles via two second-order factors . . . 33
6.2.2 Closed-form gains . . . 33
6.2.3 Numerical example and unit conversion . . . 33
6.3 Reference tracking and nudge behavior . . . 34
6.4 Implementation details (stepper-oriented) . . . 34
6.5 Parameters used in experiments . . . 35

7 Hybrid Sliding Mode Controller **36**
7.1 Nonlinear upright dynamics (acceleration input) . . . 36
7.2 Sliding surface and motion on the surface . . . 36
7.3 Surface dynamics and equivalent control . . . 36
7.4 Reaching law and complete control law . . . 37
7.5 Lyapunov stability analysis (ideal reaching law) . . . 37
7.6 Stepper-oriented implementation constraints . . . 37
7.7 Base-centering assist term (why this controller is “hybrid”) . . . 38
7.8 Parameters used in experiments . . . 38

8 Full-Surface Sliding Mode Controller (Four-State Surface) **39**
8.1 Base tracking errors . . . 39
8.2 Full-state sliding surface . . . 39
8.3 Surface dynamics and control effectiveness . . . 39
8.4 Reaching law and complete control law . . . 40
8.5 Lyapunov stability analysis (ideal reaching law) . . . 40
8.6 Implementation safeguards and stepper-friendly shaping . . . 40
8.6.1 Gated base-centering assist (implemented) . . . 41
8.7 Parameters used in experiments . . . 41

9 Experiments and Data Processing **42**
9.1 Dataset definition . . . 42
9.2 Controller parameters used in experiments . . . 42
9.3 Logging artifacts . . . 45
9.4 Event-to-CSV alignment and limitations . . . 45
9.5 Metric definitions . . . 45

10 Results and Discussion **46**
10.1 How to read this chapter . . . 46
10.2 Trials overview . . . 46
10.3 Upright hold-at-center (H1–H3) . . . 47
10.4 Commanded reference tracking / nudge mode (N1) . . . 50

10.5 Finger tap disturbance (T1–T3) 52
10.6 Overall discussion and recommendations 55

11 Conclusion and Future Work **56**
11.1 Conclusion 56
11.2 Future work 56

List of Figures

1.1 Photograph of the custom stepper-driven rotary (Furuta) inverted pendulum rig used in this work. 8
1.2 System-level signal flow for the rig. Control uses measured θ and α, but the stepper itself is still driven by open-loop STEP/DIR commands. 9
1.3 3D-printed AS5600 housing used to measure the base angle directly. The critical points are coaxial magnet alignment, rigid mounting, and consistent magnet-to-chip spacing. Small misalignment here produces poor angle data later in control experiments. 10

3.1 Rotary (Furuta) inverted pendulum schematic and generalized coordinates used in this chapter. 20

10.1 Compact comparison of the pinned hold metrics used in the discussion: α RMS, max $|\alpha|$, max $|\theta|$, and acceleration RMS. 47
10.2 Common-scale comparison panel for the pinned hold runs. Each row is one controller, and each column shows $\alpha(t)$, $\theta(t)$, or commanded acceleration. Shared scales make the controller differences easier to compare directly. 48
10.3 Phase portraits for the pinned hold runs. These plots show how tightly each controller keeps the pendulum state around upright. 48
10.4 Top row: effort histograms for the pinned hold runs. Bottom row: low-frequency α spectra. The spectra are low-frequency only (≤ 25 Hz) because the thesis logs are recorded at about 50 Hz. 49
10.5 Sparse SMC4 diagnostics taken from printed status lines in the pinned hold run. These plots help explain how the firmware protections discussed in Chapter 8 behave on the real rig. 49
10.6 Pinned linear nudge run. The target sequence, measured base angle, visible $\pm 1^\circ$ settle band, pendulum angle, and actuation effort are shown together to make the missed settling intervals easy to read. 51
10.7 Compact comparison of the pinned tap metrics used in the discussion: α RMS, max $|\alpha|$, max $|\theta|$, and acceleration RMS. 52
10.8 Common-scale comparison panel for the pinned tap runs. Shared scales make the recovery attempts and the effort differences easier to compare directly. 53
10.9 Phase portraits for the pinned tap runs. These show how much wider the disturbed state excursions are than in the hold case. 53
10.10 Top row: effort histograms for the pinned tap runs. Bottom row: low-frequency α spectra. The spectra are low-frequency only (≤ 25 Hz) because the thesis logs are recorded at about 50 Hz. 54
10.11 Sparse SMC4 diagnostics taken from printed status lines in the pinned tap run. These show how the denominator protection and ramping remain active during the disturbed recovery attempt. 54

List of Tables

1.1 Driver interface and actuation settings used in firmware. 11
1.2 Core I/O connections for the rig (Arduino Mega 2560). 12
1.3 Bill of materials (core subsystems). 13
1.4 Bill of materials (mechanical details and wiring passives). 14

6.1 Linear controller gains used in the pinned experiments (stepper acceleration units). 35

7.1 Hybrid sliding-mode controller parameters used in experiments. 38

8.1 Full-surface sliding-mode controller parameters used in experiments. 41

9.1 Dataset used for this report (manifest sessions). 42
9.2 Controller and pipeline parameters used in the pinned experimental dataset. . . . 43

10.1 Trials overview (report-facing identifiers H/N/T). 46
10.2 Summary metrics by controller mode (hold experiments). 47
10.3 Nudge/reference tracking step metrics (linear controller). 50
10.4 Summary metrics by controller mode (tap experiments). 52

Chapter 0

Problem Framing, Contributions, and Reading Guide

0.1 Problem

This project studies upright-only stabilization of a rotary (Furuta) inverted pendulum on real hardware [1, 2, 3]. The goal is simple: keep the pendulum upright while the rig deals with stepper-actuator limits, sensor noise, missed motion, and real-time timing. The base angle is measured for feedback, but the motor is still driven by open-loop STEP/DIR commands, so the actuator does not behave like an ideal servo.

0.2 Key contributions (summary)

- **Input matched to the firmware command structure.** Most Furuta pendulum treatments use motor torque as the input [4, 2]. This project instead uses an acceleration-input model, then implements that input through the firmware path "acceleration → commanded speed → stepper".
- **First-principles modelling of the custom rig.** The report derives the nonlinear model, linearizes it about the upright position, and identifies the constants used later for control design.
- **Three upright controllers on one platform.** The thesis studies one linear full-state controller, one hybrid sliding-mode controller, and one full-surface sliding-mode controller on the same rig.
- **Commanded base positioning while balancing.** A trapezoidal θ_{ref} generator with pause/resume logic allows nudge-mode reference moves. In the pinned dataset, this is demonstrated with the linear controller.
- **Practical firmware protections for a stepper-driven rig.** These include derivative filtering, wrap-safe angle differences, glitch rejection, engage logic, soft limits, stall diagnostics, and controlled disarm behavior.
- **A qualitative comparison on a fixed dataset.** Results are drawn from seven pinned runs only: H1–H3, N1, and T1–T3. The comparison is therefore qualitative and metric-based, not statistical.

0.3 Reading guide

- Hardware and constraints: Chapter 1

- Notation and units: Chapter 2
- Modelling derivation (main body): Chapter 3
- Firmware pipeline: Chapters 4–5
- Controllers: Chapters 6–8
- Experiment protocol and metric definitions: Chapter 9
- Results and comparison: Chapter 10

Chapter 1

System Hardware and Constraints

1.1 Overview

The experimental plant is a rotary (Furuta) inverted pendulum. A horizontal arm rotates about a vertical axis (base coordinate θ). The pendulum is implemented as an *L-shaped rod*: its horizontal segment forms a pivot shaft aligned with the arm (radial direction) and passes through three 688RS bearings mounted on the arm, while its vertical segment swings in the tangential–vertical plane (pendulum coordinate α). The reference configuration is *upright-only*, so $\alpha = 0$ means the pendulum is upright.

Figure 1.1 shows the custom experimental rig used in this work.

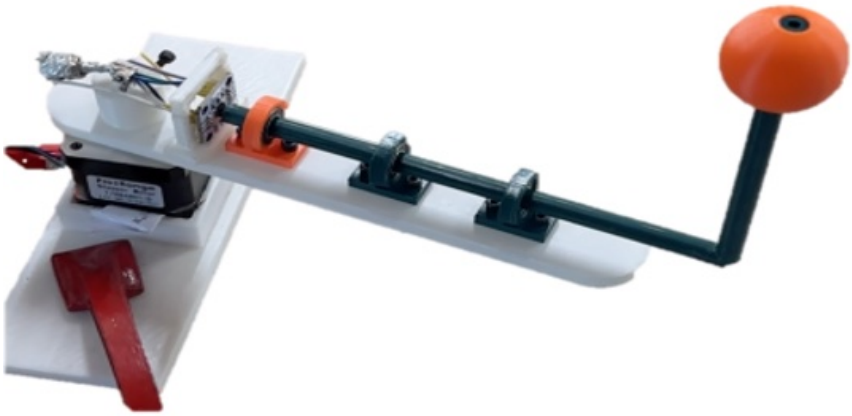


Figure 1.1: Photograph of the custom stepper-driven rotary (Furuta) inverted pendulum rig used in this work.

This report focuses on balancing near the upright position and on commanded base-position moves while the pendulum stays upright. The base angle is measured directly, but the stepper still receives open-loop STEP/DIR commands. Because of that, the controller can measure θ accurately without turning the actuator into a true torque-controlled servo.

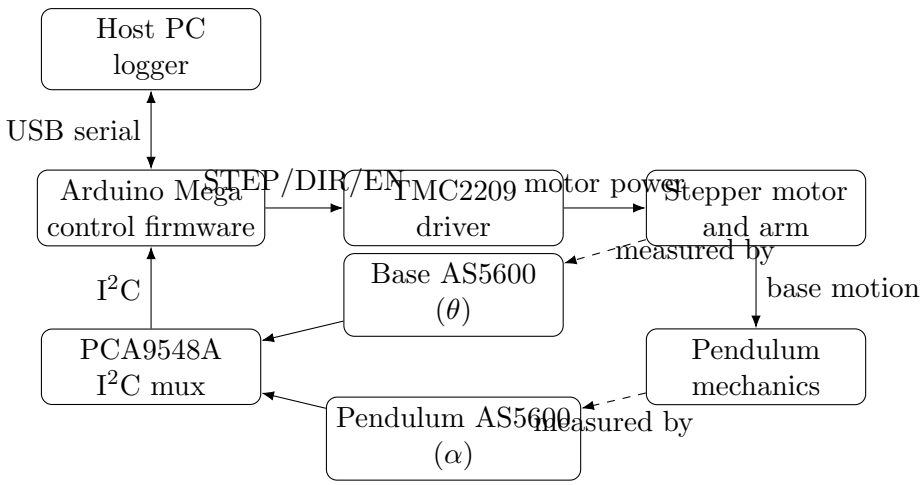


Figure 1.2: System-level signal flow for the rig. Control uses measured θ and α, but the stepper itself is still driven by open-loop STEP/DIR commands.

1.2 Hardware build-up from scratch

To document the platform clearly for student replication, the assembly was performed in the following order:

1. **Mechanical base and motor mount:** mount the stepper rigidly and verify shaft runout is acceptable before adding sensors.
2. **Rotary arm and pendulum pivot:** install the rotary arm, then align the L-shaped pendulum shaft through the three 688RS bearings.
3. **Base measurement retrofit:** turn the motor axis into a measured axis by adding the AS5600 housing and magnet assembly.
4. **Driver and power wiring:** connect STEP/DIR/EN, 24 V motor power, and common ground between motor supply, driver, and controller.
5. **I^{2}C network and noise hardening:** connect both AS5600 sensors through PCA9548A channels and apply EMI-aware routing and shielding.
6. **Calibration references:** define $\theta = 0$ and $\alpha = 0$ with multi-sample averaging before any balancing experiment.

Sections 1.3–1.5 detail each stage and the associated design constraints.

1.3 Mechanical rig

The mechanical assembly consists of:

- a rotary arm (yaw axis) driven by a stepper motor,
- an L-shaped pendulum assembly supported by a bearing-mounted pivot shaft (three 688RS bearings),
- mechanical hard-stops that bound the base rotation.

Actuation axis and coupling

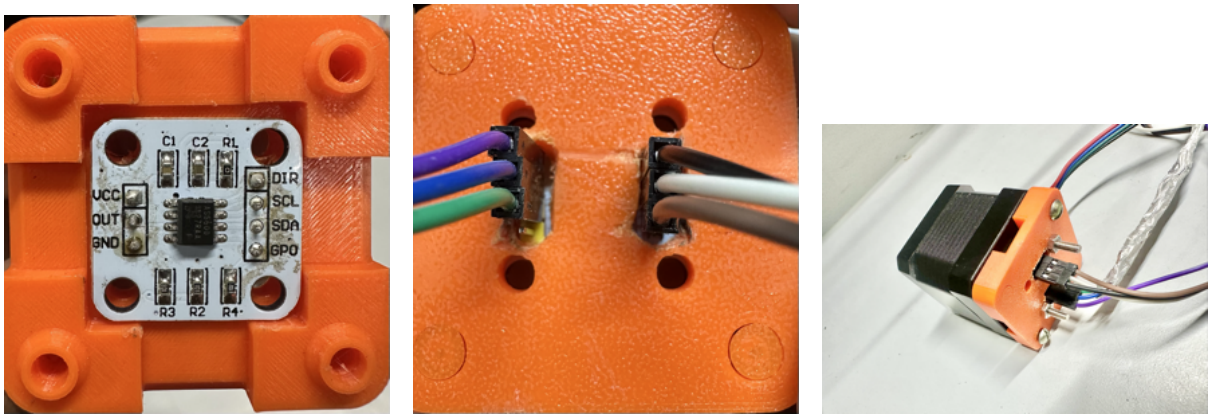
The arm is mounted coaxially to the stepper shaft (direct drive). The actuator is operated with a conversion of approximately 1600 commanded steps per arm revolution:

$$k_s \approx \frac{1600}{360} \approx 4.444\,\text{step}^{\circ -1},$$

which is the conversion used throughout the firmware and analysis.

Turning the stepper into a measured axis

In open-loop stepper operation, the commanded step count can differ from the real base angle because of missed steps, impulsive disturbances, or temporary stalls. To measure the true base motion directly, the motor axis was instrumented with a 3D-printed AS5600 housing and a coaxial magnet mount. This improves state measurement, but it does not remove the possibility of missed motion under load.



(a) Front view of housing with sensor installed.

(b) Back view showing mounting geometry.

(c) Housing fixed to the motor base.

Figure 1.3: 3D-printed AS5600 housing used to measure the base angle directly. The critical points are coaxial magnet alignment, rigid mounting, and consistent magnet-to-chip spacing. Small misalignment here produces poor angle data later in control experiments.

Pendulum pivot (bearing-supported)

The pendulum rotates about a bearing-supported pivot shaft that is aligned with the rotary arm. The shaft passes through three 688RS bearings, which reduce friction and minimize backlash in the pendulum degree of freedom. This matters because pivot losses couple directly into the base acceleration required for upright stabilization. The pivot assembly also carries the pendulum angle encoder magnet and fixture so the AS5600 measures α directly about this axis.

Key geometric parameters

The main geometric and inertial parameters are derived and numerically identified in Chapter 3. For reference, the rig uses:

- arm length $L_r \approx 0.19\,\text{m}$,
- pendulum vertical length $L_v \approx 0.12\,\text{m}$ with a concentrated mass near the lower end.

1.4 Electronics and actuation

Controller and driver

The base actuator is a stepper motor controlled by:

- an Arduino Mega 2560 microcontroller (real-time control loop),
- a TMC2209 stepper driver (STEP/DIR/EN interface),
- a dedicated motor supply (24 V) separate from the Arduino 5 V logic rail.

Motor power, driver, and microcontroller share a common ground. The driver supply is decoupled using a bulk electrolytic capacitor and a small ceramic capacitor close to the driver.

Driver and actuation settings

Table 1.1: Driver interface and actuation settings used in firmware.

Setting	Value	Notes
STEP/DIR/EN pins	D11 / D6 / D7	Digital control outputs to the TMC2209 driver.
I^2C bus clock	400 kHz	Multiplexer and AS5600 sensors share this bus speed.
Base conversion	1600 steps/rev ($4.444\,\mathrm{step}\,^{\circ -1}$)	Conversion used in firmware for actuation and analysis.

Acceleration-input actuation model

The driver is commanded in continuous-velocity mode (step rate in $\mathrm{step\,s^{-1}}$). To match this actuation pipeline, all controllers in this project are written in an *acceleration-input form*: the controller computes a commanded base acceleration (in $\mathrm{step\,s^{-2}}$), which is integrated in firmware to produce the commanded velocity:

$$u \equiv \ddot{\theta} \;\longrightarrow\; \dot{\theta}_{\mathrm{cmd}} \;\longrightarrow\; \text{stepper speed command.}$$

This is a useful design match between the model and the firmware command structure. It does *not* mean the hardware realizes the commanded acceleration exactly, because the stepper still receives an open-loop speed command and can miss motion under load.

1.5 Sensing

Angle sensing

Two AS5600 magnetic encoders measure absolute angles. Both devices share the same fixed I^2C address, so they are read through an I^2C multiplexer (PCA9548A, default address 0x70) at 400 kHz:

- pendulum encoder (at the pendulum pivot axis),
- base encoder (at the motor/base axis, see Figure 1.3).

Each AS5600 provides a 12 bit absolute angle measurement over $[0, 360^\circ)$ (nominal resolution $\approx 0.0879^\circ$ per count).

Mechanical encoder mounting details

Both pendulum and base sensors use:

- a rigid printed housing to locate the AS5600 PCB relative to the rotating axis,
- a coaxial magnet mount attached to the rotating part,
- a controlled magnet-to-chip air-gap (typically 2 mm to 3 mm),
- mechanical features to reduce radial offset and wobble.

This reduces sensitivity to small misalignments that can otherwise cause jitter, dropouts, or inconsistent angle readings. The firmware also monitors AS5600 magnet-status diagnostics (detect/high/low) during some fault conditions to help distinguish true actuator faults from sensor misalignment.

I^2C interference mitigation used in practice

Because AS5600 links run close to a high-current stepper drive, the I^2C bus was treated as an EMI-sensitive interface. The baseline wiring rules used in this build were:

- twist SDA and SCL together tightly and keep the run short,
- route I^2C wires away from motor phase wires and driver power loops,
- wrap the twisted pair with aluminium foil to create a simple shield,
- connect the foil shield to ground at the controller side only, leaving the sensor-side foil end floating.

This hardware practice reduced burst-like glitches caused by switching noise. Software-side protections (jump rejection and derivative-safe handling) are documented in Chapter 4, Section 4.3.

Calibration references

Before each experimental trial, a short calibration routine averages multiple sensor samples (default: 25) to reduce single-sample noise and define reference angles:

- $\alpha = 0$ reference: pendulum held upright,
- $\theta = 0$ reference: arm centered.

Pin assignments (summary)

For completeness, the essential microcontroller I/O used by the rig is:

Table 1.2: Core I/O connections for the rig (Arduino Mega 2560).

Function	Arduino pins	Notes
Stepper driver signals (STEP, DIR, EN)	D11 (STEP), D6 (DIR), D7 (EN)	digital outputs to the stepper driver
I^2C bus (SDA/SCL)	D20 (SDA), D21 (SCL)	shared by PCA9548A and both AS5600 encoders
I^2C multiplexer channels	ch0 (pendulum), ch1 (base)	selects which AS5600 encoder is visible on the bus

1.6 Bill of materials

Tables 1.3 and 1.4 list the hardware required to reproduce the rig. The functional design is fixed, but a few procurement details were not finalized when this report was written. Those unresolved entries are kept visible instead of being guessed.

Table 1.3: Bill of materials (core subsystems).

Item	Qty	Spec/Model	Purpose	Notes/Source
Microcontroller board	1	Arduino Mega 2560	Real-time control and serial interface	Matches firmware pin map and loop implementation
Stepper driver module	1	TMC2209 (STEP/DIR/EN)	Stepper current control and pulse interface	Base axis driver stage
Stepper motor	1	1.8°/step, model `TBD`	Base rotational actuation	Exact model/vendor `TBD`
Motor power supply	1	24 V DC, current `TBD`	Dedicated motor rail	Repo indicates 24 V; adapter model `TBD`
Magnetic encoder modules	2	AS5600	Absolute pendulum and base angle sensing	One on pendulum, one on base axis
I^2C multiplexer	1	PCA9548A (0x70 default)	Allows two AS5600 devices on one I^2C bus	ch0 pendulum, ch1 base
Encoder magnets	2	Diametric magnet, size `TBD`	Magnetic target for AS5600	Maintain 2 mm to 3 mm air gap

1.7 Host computer and data acquisition

All experiments are run with a host computer connected over USB serial. A Python logger records:

- a device-time CSV stream (angles, angular rates, and command signals),
- a text event stream (status lines and diagnostic messages).

These artifacts are later used to compute metrics and generate the figures and tables in the Results chapter.

1.8 Safety limits and operating envelope

The rig is operated under explicit safety limits for repeatability:

- Base limit: $|\theta| > 80°$ triggers a safety disarm.
- Pendulum limit: $|\alpha| > 30°$ triggers a safety disarm.
- Upright-only validity window (nonlinear controllers): an additional hard abort is applied at $|\alpha| > 25°$ (and also at extreme $|\dot{\alpha}|$) to enforce upright-only operation and avoid numerically unsafe regions.

Limit checks are debounced across multiple control ticks to avoid false positives from single-sample sensor glitches. When a limit is confirmed, the motor output is disabled and the system returns to an idle state so the trial can be restarted safely.

Table 1.4: Bill of materials (mechanical details and wiring passives).

Item	Qty	Spec/Model	Purpose	Notes/Source
Pendulum pivot bearings	3	688RS	Low-friction pendulum pivot support	Bearing-supported L-pivot shaft
Rotary arm assembly	1	Custom part, $L_r \approx 0.19\,\mathrm{m}$	Base arm geometry	Fabrication details `TBD`
L-shaped pendulum assembly	1	Custom part, $L_v \approx 0.12\,\mathrm{m}$	Pendulum body and mass distribution	Includes pivot shaft and end mass
Base encoder 3D-printed housing set	1 set	Custom CAD print (`sensor_housing_*`)	Converts motor axis into a measured axis	See Figure 1.3
Pull-up resistors	2	$5\,\mathrm{k\Omega}$, tolerance `TBD`	SDA/SCL biasing	Final resistor package/tolerance `TBD`
Driver bulk capacitor	1	$100\,\mu\mathrm{F}$, voltage `TBD`	VMOT rail smoothing	Place close to VMOT/GND pins
Driver/sensor ceramic capacitors	`TBD`	$0.1\,\mu\mathrm{F}$, package `TBD`	High-frequency decoupling	One near VMOT and one per sensor supply
Sensor local capacitors	2	$1\,\mu\mathrm{F}$, voltage `TBD`	Local AS5600 supply filtering	Final dielectric/package `TBD`
Prototyping and wiring hardware	`TBD`	Breadboards, jumpers, foil shield, connectors	Electrical assembly and EMI mitigation	Counts depend on physical layout

1.9 Replication notes and lessons learned

The most common bring-up failures in this project were sensor noise spikes, magnet or housing misalignment, I^2C interference from motor wiring, and same-address encoder conflicts without multiplexing. The most effective troubleshooting order was: establish rigid mechanical alignment first, verify magnet spacing and status bits, apply twisted-and-shielded I^2C wiring with one-end shield grounding, and then enable software glitch rejection and calibration averaging. Following this order reduces repeated debugging loops and makes student replication faster.

Chapter 2

Notation, Units, and Recorded Signals

2.1 Coordinates

The system is described by two angles:

- $\theta(t)$: base or arm yaw angle about the motor axis,
- $\alpha(t)$: pendulum angle about the bearing-supported pivot axis, measured from the upright configuration ($\alpha = 0$ upright).

The state vector used by all controllers is

$$\mathbf{x}(t) = \begin{bmatrix} \theta & \alpha & \dot{\theta} & \dot{\alpha} \end{bmatrix}^{\mathsf{T}}.$$

2.2 Sign conventions

Throughout this report, the sign conventions are chosen such that:

- positive θ corresponds to one fixed arm rotation direction when viewed from above,
- positive α corresponds to one fixed pendulum tilt direction relative to the arm.

In practice, different builds can differ in wiring and sensor mounting. The sign settings are therefore checked experimentally during bring-up: a small manual pendulum tilt is used to verify the sign of α, and a small commanded motor motion is used to verify the signs of θ and the motor command. For the pinned thesis dataset, the saved settings were `motorSign = 1`, `ALPHA_SIGN = -1`, `THETA_SIGN = 1`, and `CTRL_SIGN = 1`. The modelling and controller equations in this report assume this sign consistency.

2.3 Wrap-safe angle differences

Both encoders provide absolute angles in $[0, 360^\circ)$. For control and reference tracking we use the shortest signed angle difference:

$$\mathrm{diff}(a, b) \in [-180^\circ, 180^\circ], \qquad \mathrm{diff}(a, b) \equiv a - b \text{ wrapped to } [-180^\circ, 180^\circ].$$

This avoids discontinuities at $0/360^\circ$. It is used, for example, for the base tracking error

$$\theta_{\mathrm{err}} = \mathrm{diff}(\theta, \theta_{\mathrm{ref}}).$$

In firmware this means *current minus reference*, matching the helper `getAngleDiffDeg`.

2.4 Units policy and stepper conversion

The first-principles modelling in Chapter 3 is expressed in SI units (radians, seconds). The embedded implementation and plots in this report use degrees for angles for readability.

The stepper driver is commanded in $\mathrm{step\,s^{-1}}$. A constant conversion between angle and steps is used:

$$k_s \equiv \frac{\text{steps}}{\text{degree}},$$

so that θ can be converted to measured steps by $k_s\,\theta$. Likewise, an angular acceleration command ($^\circ\,\mathrm{s}^{-2}$) can be converted to a stepper acceleration command ($\mathrm{step\,s^{-2}}$) by multiplying by k_s.

When this report writes $u = \ddot{\theta}$ in the model chapters, u is the abstract acceleration input. In firmware and plots, the logged command is the implementation-space version of that input, expressed in stepper units.

2.5 Variables and recorded signals

This report uses three related but different naming layers:

- **Model variables:** θ, α, $\dot{\theta}$, $\dot{\alpha}$, and $u = \ddot{\theta}$.
- **Firmware variables:** names used directly in the embedded code.
- **Raw CSV columns:** the columns written directly by the logger.

Model variables

- θ, α: measured base and pendulum angles,
- $\dot{\theta}$, $\dot{\alpha}$: estimated angular rates,
- $u = \ddot{\theta}$: the control input used in the model and control derivations.

Firmware variables

- `thetaErr = diff(theta, thetaRef)`: base tracking error,
- `thetaDotErr = thetaDot - thetaRefDot`: base-rate tracking error,
- `accCmdPhysical`: commanded base acceleration before motor-sign mapping,
- `thetaDotCmdMotor`: commanded motor speed after acceleration integration.

Raw CSV columns

Each experimental trial records:

- `alpha_deg`, `alpha_dot_deg_s`,
- `theta_deg`, `theta_dot_deg_s`,
- `acc_cmd_steps_s2`, `vel_cmd_steps_s`,
- `pos_meas_steps`, `position_clamped`.

The raw CSV does *not* contain a dedicated acceleration-saturation column. Saturation percentages are computed later by the analysis pipeline from the logged command and the known firmware limits.

In addition to the high-rate CSV stream, the firmware also prints low-rate status and diagnostic lines used to segment trials and, when possible, align host-time events to device-time signals (Chapter 9).

Chapter 3

Mathematical Modelling from First Principles

This chapter derives the nonlinear dynamics of the rotary (Furuta) inverted pendulum from first principles. The goal is to obtain a simple model that can be used for parameter identification and upright control design. The model in this chapter is ideal: it does not include missed steps, sensor glitches, or actuator saturation. Those real-hardware effects are handled later in the firmware chapters.

The objective is to obtain:

- numerical parameter identification for the custom rig,
- controller design about the upright equilibrium.

This chapter builds the mathematical model used later for controller design. The main outputs are:

- the nonlinear upright model,
- the linearized upright model,
- the numerical constants used later in the firmware and controller chapters.

The goal here is not to model every real-world effect. The model is intentionally idealized so the main pendulum physics can be written clearly.

Assumptions used in this chapter:

- rigid links,
- frictionless joints,
- commanded base acceleration $\ddot{\theta}$ treated as the input,
- no missed steps or motor stalls,
- no sensor glitches,
- no actuator saturation.

These assumptions are good for control design, but they do not describe the whole real platform. Later firmware chapters add the practical protections needed for the real stepper-driven rig.

3.1 Definition

This section defines the coordinates and grouped constants used in the derivation.

Generalized coordinates:

$\theta(t)$: arm yaw angle.

$\alpha(t)$: pendulum angle, with $\alpha = 0$ upright.

Constants:

L_r: arm length (motor axis to pendulum pivot axis).

L_h: horizontal length of the L-shaped pendulum (pivot shaft segment).

L_v: vertical length of the L-shaped pendulum (swinging segment).

m_p: swinging mass.

l_p: distance from pivot axis to pendulum COM (of the swinging mass).

g: gravity.

τ: actuation torque about θ.

Geometry note (this rig): the pendulum is an **L-shaped rod**. The horizontal segment is aligned with the arm (radial direction) and forms a bearing-supported pivot shaft (three 688RS bearings). The vertical segment carries most of the swinging mass and swings in the $(\mathbf{e}_t, \mathbf{k})$ plane.

In the reduced model used for control, m_p and l_p refer to the mass distribution that moves with α (vertical segment plus concentrated mass). The horizontal segment lies on the pivot axis and contributes primarily to arm-side inertia.

Define

$$K \equiv m_p L_r l_p, \qquad G \equiv m_p g l_p$$

These grouped constants are introduced for compactness: G appears when collecting gravity potential/torque terms, and K appears when collecting inertial coupling terms between arm rotation and pendulum swing.

Let

$\hat{J}_1$: yaw inertia about motor axis (arm-side).

$\hat{J}_2$: pendulum pivot inertia (about the pivot axis).

J_p: pendulum inertia about its center of mass, about an axis perpendicular to the swing plane.

$$\hat{J}_0 \triangleq \hat{J}_1 + m_p L_r^2$$

The definition of $\hat{J}_0$ follows from collecting the $\dot{\theta}^2$ terms in the kinetic energy: it is the arm-side yaw inertia about the motor axis plus the parallel-axis contribution of the swinging mass at radius L_r.

A schematic of the coordinates and parameters is shown in Fig. 3.1.

3.2 Parameter evaluation

This section substitutes the measured dimensions and masses of the real rig into the symbols defined above. These numerical values are later used to compute the final model constants.

Formal parameter identification methods for Furuta pendulum rigs (including fitting inertia/friction terms from data) are well documented; see, e.g., [5].

Given:

$$L_r = 0.19 \text{ m}, \quad m_r = 0.051 \text{ kg}$$

$$L_h = 0.17 \text{ m}, \quad L_v = 0.12 \text{ m}$$

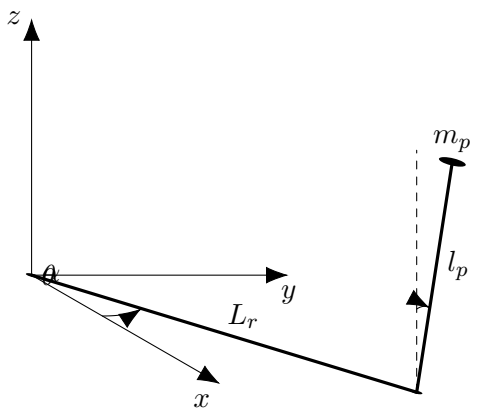


Figure 3.1: Rotary (Furuta) inverted pendulum schematic and generalized coordinates used in this chapter.

$$m_{\text{rod,total}} = 10.3 \text{ g} = 0.0103 \text{ kg}$$

$$m_s = 7.7 \text{ g} = 0.0077 \text{ kg}, \quad x_s = 0.103 \text{ m}$$

Linear density:

$$\lambda = \frac{10.3 \text{ g}}{290 \text{ mm}} = 0.03552 \text{ g/mm}$$

Mass split:

$$m_h = 170\lambda = 6.04 \text{ g} = 0.006038 \text{ kg}$$

$$m_{\text{rod}} = 120\lambda = 4.26 \text{ g} = 0.004262 \text{ kg}$$

Swinging mass:

$$m_p = m_{\text{rod}} + m_s = 0.011962069 \text{ kg}$$

COM distance:

$$x_{\text{rod}} = \frac{L_v}{2} = 0.06 \text{ m}$$

$$l_p = \frac{m_{\text{rod}} x_{\text{rod}} + m_s x_s}{m_p}$$

$$l_p = \frac{(0.004262)(0.06) + (0.0077)(0.103)}{0.011962069}$$

$$l_p = \frac{2.5572 \times 10^{-4} + 7.931 \times 10^{-4}}{0.011962069} = 0.087679158 \text{ m}$$

Pendulum pivot inertia (rod about end + sphere point mass):

$$I_{\text{rod,piv}} = \frac{1}{3} m_{\text{rod}} L_v^2 = \frac{1}{3}(0.004262)(0.12^2) = 2.045793103 \times 10^{-5}$$

$$I_{\text{s,piv}} = m_s x_s^2 = (0.0077)(0.103^2) = 8.16893 \times 10^{-5}$$

$$\hat{J}_2 = I_{\text{rod,piv}} + I_{\text{s,piv}} = 1.021472310 \times 10^{-4} \text{ kg m}^2$$

Gravity constant:

$$G = m_p g l_p = (0.011962069)(9.81)(0.087679158) = 1.028896479 \times 10^{-2} \text{ N m}$$

Coupling constant:

$$K = m_p L_r l_p = (0.011962069)(0.19)(0.087679158) = 1.992765862 \times 10^{-4}\ \mathrm{kg\,m^2}$$

Arm-side yaw inertia:

$$\hat{J}_0 = \hat{J}_1 + m_p L_r^2$$

At this stage we have the geometric and inertia terms needed to write the kinematics and energy equations.

3.3 Kinematics

The next step is to write the pendulum center-of-mass position and velocity in terms of θ and α. This is where the coupling between arm rotation and pendulum swing first appears.

Let $\{\mathbf{i}, \mathbf{j}, \mathbf{k}\}$ be an inertial orthonormal basis. Define the planar arm basis

$$\mathbf{e}_r \equiv \cos\theta\,\mathbf{i} + \sin\theta\,\mathbf{j}, \qquad \mathbf{e}_t \equiv -\sin\theta\,\mathbf{i} + \cos\theta\,\mathbf{j}$$

Pendulum pivot position:

$$\mathbf{r}_h = L_r \mathbf{e}_r$$

Pendulum COM relative to pivot:

$$\mathbf{r}_{p/h} = l_p\big(\sin\alpha\,\mathbf{e}_t + \cos\alpha\,\mathbf{k}\big)$$

Pendulum COM position:

$$\mathbf{r}_p = \mathbf{r}_h + \mathbf{r}_{p/h}$$

Components $\mathbf{r}_p = x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$:

$$x = L_r \cos\theta - l_p \sin\alpha\,\sin\theta$$
$$y = L_r \sin\theta + l_p \sin\alpha\,\cos\theta$$
$$z = l_p \cos\alpha$$

Derivatives:

$$\frac{d}{dt}(\cos\theta) = -\sin\theta\,\dot{\theta}, \quad \frac{d}{dt}(\sin\theta) = \cos\theta\,\dot{\theta}$$
$$\frac{d}{dt}(\sin\alpha) = \cos\alpha\,\dot{\alpha}, \quad \frac{d}{dt}(\cos\alpha) = -\sin\alpha\,\dot{\alpha}$$

3.3.1 Velocity components

We first differentiate the position coordinates one by one.

For x:

$$\dot{x} = \frac{d}{dt}(L_r \cos\theta) - l_p \frac{d}{dt}(\sin\alpha\ \sin\theta)$$

$$\frac{d}{dt}(L_r \cos\theta) = -L_r \sin\theta\,\dot{\theta}$$

$$\frac{d}{dt}(\sin\alpha\ \sin\theta) = (\cos\alpha\,\dot{\alpha})\sin\theta + \sin\alpha(\cos\theta\,\dot{\theta})$$

$$\dot{x} = -L_r \sin\theta\,\dot{\theta} - l_p \cos\alpha \sin\theta\,\dot{\alpha} - l_p \sin\alpha \cos\theta\,\dot{\theta}$$

For y:

$$\dot{y} = \frac{d}{dt}(L_r \sin\theta) + l_p \frac{d}{dt}(\sin\alpha\ \cos\theta)$$

$$\frac{d}{dt}(L_r \sin\theta) = L_r \cos\theta\,\dot{\theta}$$

$$\frac{d}{dt}(\sin\alpha\ \cos\theta) = (\cos\alpha\,\dot{\alpha})\cos\theta + \sin\alpha(-\sin\theta\,\dot{\theta})$$

$$\dot{y} = L_r \cos\theta\,\dot{\theta} + l_p \cos\alpha \cos\theta\,\dot{\alpha} - l_p \sin\alpha \sin\theta\,\dot{\theta}$$

For z:

$$\dot{z} = \frac{d}{dt}(l_p \cos\alpha) = -l_p \sin\alpha\,\dot{\alpha}$$

3.3.2 Speed squared

Now combine the velocity components into one expression for v^2. This quantity is needed for the translational kinetic energy.

Compute $v^2 = \dot{x}^2 + \dot{y}^2 + \dot{z}^2$.

Define

$$A = L_r \sin\theta + l_p \sin\alpha \cos\theta, \quad B = l_p \cos\alpha \sin\theta$$

$$C = L_r \cos\theta - l_p \sin\alpha \sin\theta, \quad D = l_p \cos\alpha \cos\theta$$

Then

$$\dot{x} = -A\dot{\theta} - B\dot{\alpha}, \qquad \dot{y} = C\dot{\theta} + D\dot{\alpha}, \qquad \dot{z} = -l_p \sin\alpha\,\dot{\alpha}$$

Square:

$$\dot{x}^2 = A^2\dot{\theta}^2 + B^2\dot{\alpha}^2 + 2AB\dot{\theta}\dot{\alpha}$$

$$\dot{y}^2 = C^2\dot{\theta}^2 + D^2\dot{\alpha}^2 + 2CD\dot{\theta}\dot{\alpha}$$

$$\dot{z}^2 = l_p^2 \sin^2\alpha\,\dot{\alpha}^2$$

Collect $\dot{\theta}^2$:

$$A^2 + C^2 = (L_r \sin\theta + l_p \sin\alpha \cos\theta)^2 + (L_r \cos\theta - l_p \sin\alpha \sin\theta)^2$$

Expand:

$$A^2 = L_r^2 \sin^2\theta + l_p^2 \sin^2\alpha \cos^2\theta + 2L_r l_p \sin\alpha \sin\theta \cos\theta,$$
$$C^2 = L_r^2 \cos^2\theta + l_p^2 \sin^2\alpha \sin^2\theta - 2L_r l_p \sin\alpha \sin\theta \cos\theta.$$

Add:

$$A^2 + C^2 = L_r^2(\sin^2\theta + \cos^2\theta) + l_p^2 \sin^2\alpha(\sin^2\theta + \cos^2\theta) = L_r^2 + l_p^2 \sin^2\alpha$$

Collect $\dot{\alpha}^2$:

$$B^2 + D^2 + l_p^2 \sin^2\alpha = l_p^2 \cos^2\alpha(\sin^2\theta + \cos^2\theta) + l_p^2 \sin^2\alpha = l_p^2$$

Collect $\dot{\theta}\dot{\alpha}$:

$$AB + CD = l_p \cos\alpha \Big[(L_r \sin\theta + l_p \sin\alpha \cos\theta)\sin\theta + (L_r \cos\theta - l_p \sin\alpha \sin\theta)\cos\theta\Big]$$

Expand bracket:

$$\begin{aligned}&(L_r \sin\theta + l_p \sin\alpha \cos\theta)\sin\theta + (L_r \cos\theta - l_p \sin\alpha \sin\theta)\cos\theta \\ &= L_r(\sin^2\theta + \cos^2\theta) + l_p \sin\alpha(\cos\theta \sin\theta - \sin\theta \cos\theta) = L_r.\end{aligned}$$

Thus

$$AB + CD = L_r l_p \cos\alpha$$

Therefore

$$v^2 = (L_r^2 + l_p^2 \sin^2\alpha)\dot{\theta}^2 + l_p^2 \dot{\alpha}^2 + 2L_r l_p \cos\alpha\, \dot{\theta}\dot{\alpha}$$

3.4 Energies

Once the kinematics are known, the energy expressions are straightforward: translational kinetic energy, rotational kinetic energy, and gravitational potential energy.

Translational kinetic energy:

$$T_{\text{trans}} = \frac{1}{2} m_p v^2$$

Rotational kinetic energy of the pendulum about its COM (perpendicular axis):

$$T_{\text{rot}} = \frac{1}{2} J_p(\dot{\alpha}^2 + \sin^2\alpha\, \dot{\theta}^2)$$

Parallel axis:

$$\hat{J}_2 = J_p + m_p l_p^2$$

Arm yaw kinetic energy:

$$T_{\text{arm}} = \frac{1}{2} \hat{J}_1 \dot{\theta}^2$$

Total kinetic energy:

$$T = T_{\text{arm}} + T_{\text{trans}} + T_{\text{rot}}$$

Collect terms.
Coefficient of $\dot{\theta}^2$:

$$\frac{1}{2}\Big[\hat{J}_1 + m_p L_r^2 + (m_p l_p^2 + J_p)\sin^2\alpha\Big]\dot{\theta}^2 = \frac{1}{2}(\hat{J}_0 + \hat{J}_2\sin^2\alpha)\dot{\theta}^2$$

Coefficient of $\dot{\alpha}^2$:

$$\frac{1}{2}(m_p l_p^2 + J_p)\dot{\alpha}^2 = \frac{1}{2}\hat{J}_2\dot{\alpha}^2$$

Cross term:

$$m_p L_r l_p \cos\alpha\,\dot{\theta}\dot{\alpha} = K\cos\alpha\,\dot{\theta}\dot{\alpha}$$

Thus

$$T = \frac{1}{2}(\hat{J}_0 + \hat{J}_2\sin^2\alpha)\dot{\theta}^2 + \frac{1}{2}\hat{J}_2\dot{\alpha}^2 + K\cos\alpha\,\dot{\theta}\dot{\alpha}$$

Potential energy:

$$V = m_p g z = m_p g l_p \cos\alpha = G\cos\alpha$$

Lagrangian:

$$\mathcal{L} = T - V$$

3.5 Euler–Lagrange equations

The Euler–Lagrange equations convert the energy expressions into equations of motion for the two generalized coordinates.

$$\frac{d}{dt}\left(\frac{\partial\mathcal{L}}{\partial\dot{q}}\right) - \frac{\partial\mathcal{L}}{\partial q} = Q$$

$$Q_\theta = \tau, \qquad Q_\alpha = 0$$

3.5.1 Equation in θ

This is the arm or base-axis equation.

$$\frac{\partial\mathcal{L}}{\partial\dot{\theta}} = (\hat{J}_0 + \hat{J}_2\sin^2\alpha)\dot{\theta} + K\cos\alpha\,\dot{\alpha}$$

Differentiate:

$$\frac{d}{dt}\Big[(\hat{J}_0 + \hat{J}_2\sin^2\alpha)\dot{\theta}\Big] = (\hat{J}_0 + \hat{J}_2\sin^2\alpha)\ddot{\theta} + \hat{J}_2\frac{d}{dt}(\sin^2\alpha)\dot{\theta}$$

$$\frac{d}{dt}(\sin^2\alpha) = 2\sin\alpha\cos\alpha\,\dot{\alpha} = \sin(2\alpha)\dot{\alpha}$$

$$\frac{d}{dt}(K\cos\alpha\,\dot{\alpha}) = K\left(-\sin\alpha\,\dot{\alpha}^2 + \cos\alpha\,\ddot{\alpha}\right)$$

Also $\partial\mathcal{L}/\partial\theta = 0$.
Thus

$$(\hat{J}_0 + \hat{J}_2\sin^2\alpha)\ddot{\theta} + K\cos\alpha\,\ddot{\alpha} + \hat{J}_2\sin(2\alpha)\dot{\theta}\dot{\alpha} - K\sin\alpha\,\dot{\alpha}^2 = \tau$$

3.5.2 Equation in α

This is the pendulum equation.

$$\frac{\partial \mathcal{L}}{\partial \dot{\alpha}} = \hat{J}_2 \dot{\alpha} + K \cos\alpha\, \dot{\theta}$$

Differentiate:

$$\frac{d}{dt}\left(\hat{J}_2 \dot{\alpha}\right) = \hat{J}_2 \ddot{\alpha}$$

$$\frac{d}{dt}(K \cos\alpha\, \dot{\theta}) = K\left(-\sin\alpha\, \dot{\alpha}\dot{\theta} + \cos\alpha\, \ddot{\theta}\right)$$

Partial derivative:

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \frac{1}{2}\hat{J}_2 \frac{\partial}{\partial \alpha}(\sin^2\alpha)\dot{\theta}^2 - K \sin\alpha\, \dot{\theta}\dot{\alpha} + G \sin\alpha$$

$$\frac{\partial}{\partial \alpha}(\sin^2\alpha) = \sin(2\alpha)$$

Thus

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \frac{1}{2}\hat{J}_2 \sin(2\alpha)\dot{\theta}^2 - K \sin\alpha\, \dot{\theta}\dot{\alpha} + G \sin\alpha$$

Euler–Lagrange ($Q_\alpha = 0$):

$$\hat{J}_2 \ddot{\alpha} + K \cos\alpha\, \ddot{\theta} - K \sin\alpha\, \dot{\alpha}\dot{\theta} - \left(\frac{1}{2}\hat{J}_2 \sin(2\alpha)\dot{\theta}^2 - K \sin\alpha\, \dot{\theta}\dot{\alpha} + G \sin\alpha\right) = 0$$

Cancel the $\pm K \sin\alpha\, \dot{\theta}\dot{\alpha}$ terms:

$$K \cos\alpha\, \ddot{\theta} + \hat{J}_2 \ddot{\alpha} - \frac{1}{2}\hat{J}_2 \sin(2\alpha)\dot{\theta}^2 - G \sin\alpha = 0$$

3.6 Nonlinear model

Combining the two Euler–Lagrange equations gives the nonlinear model used later for nonlinear controller design:

$$(\hat{J}_0 + \hat{J}_2 \sin^2\alpha)\ddot{\theta} + K \cos\alpha\, \ddot{\alpha} + \hat{J}_2 \sin(2\alpha)\dot{\theta}\dot{\alpha} - K \sin\alpha\, \dot{\alpha}^2 = \tau$$

$$K \cos\alpha\, \ddot{\theta} + \hat{J}_2 \ddot{\alpha} - \frac{1}{2}\hat{J}_2 \sin(2\alpha)\dot{\theta}^2 - G \sin\alpha = 0$$

These equations are structurally consistent with classical Furuta pendulum models [4]. The key difference in this thesis is that later controller chapters use commanded base acceleration as the abstract input, because that matches the firmware command structure more naturally than a motor-torque input. This does *not* mean the real hardware realizes the commanded acceleration exactly; on the real rig the command is still turned into a speed command and the stepper can miss motion under load.

3.7 Linearization about upright ($\alpha \approx 0$)

For upright control design, the nonlinear model is simplified around the operating point $\alpha \approx 0$. This keeps the main unstable pendulum physics while making the equations much easier to use in controller design.

$$\sin\alpha \approx \alpha, \qquad \cos\alpha \approx 1, \qquad \sin(2\alpha) \approx 2\alpha$$

Neglect products of small quantities.
Linear model:

$$\hat{J}_0\ddot{\theta} + K\ddot{\alpha} = \tau$$
$$K\ddot{\theta} + \hat{J}_2\ddot{\alpha} - G\alpha = 0$$

Solve for $\ddot{\alpha}$:

$$\ddot{\alpha} = \frac{G}{\hat{J}_2}\alpha - \frac{K}{\hat{J}_2}\ddot{\theta}$$

3.8 Numerical specialization

This section substitutes the measured rig parameters into the linearized model so the final constants used later in the thesis can be read directly.

Let $J_0 \equiv \hat{J}_0$ and $J_1 \equiv \hat{J}_2$. Numerical values:

$$J_0 = 0.001104\ \mathrm{kg\,m^2}, \quad J_1 = 1.021\times10^{-4}\ \mathrm{kg\,m^2}, \quad K = 1.993\times10^{-4}\ \mathrm{kg\,m^2}, \quad G = 0.01029\ \mathrm{N\,m}$$

Linear equations:

$$J_0\ddot{\theta} + K\ddot{\alpha} = \tau$$
$$K\ddot{\theta} + J_1\ddot{\alpha} - G\alpha = 0$$

Reduced relation:

$$\ddot{\alpha} = \frac{G}{J_1}\alpha - \frac{K}{J_1}\ddot{\theta}$$

Numerically:

$$\ddot{\alpha} = 100.8\,\alpha - 1.952\,\ddot{\theta}$$

Open-loop fall rate ($\tau = 0$). From $\ddot{\theta} = -(K/J_0)\ddot{\alpha}$:

$$\left(J_1 - \frac{K^2}{J_0}\right)\ddot{\alpha} - G\alpha = 0$$

$$J_{eq} = J_1 - \frac{K^2}{J_0} = 0.0001021 - \frac{(0.0001993)^2}{0.001104} = 6.612\times10^{-5}$$

$$\ddot{\alpha} = \frac{G}{J_{eq}}\alpha, \qquad \lambda = \pm\sqrt{\frac{G}{J_{eq}}} = \pm12.47\ \mathrm{rad/s}$$

Mass matrix inverse:

$$M = \begin{bmatrix} J_0 & K \\ K & J_1 \end{bmatrix}, \qquad M^{-1} = \frac{1}{J_0 J_1 - K^2} \begin{bmatrix} J_1 & -K \\ -K & J_0 \end{bmatrix}$$

$$J_0 J_1 - K^2 = 7.2998 \times 10^{-8}$$

$$M^{-1} = \begin{bmatrix} 1398.7 & -2730.2 \\ -2730.2 & 15123.7 \end{bmatrix}$$

Substituted form:

$$0.001104\,\ddot{\theta} + 0.0001993\,\ddot{\alpha} = \tau$$
$$0.0001993\,\ddot{\theta} + 0.0001021\,\ddot{\alpha} - 0.01029\,\alpha = 0$$

Nonlinear model with numerical constants:

$$(J_0 + J_1 \sin^2 \alpha)\ddot{\theta} + K \cos\alpha\,\ddot{\alpha} + J_1 \sin(2\alpha)\dot{\theta}\dot{\alpha} - K \sin\alpha\,\dot{\alpha}^2 = \tau$$

$$K \cos\alpha\,\ddot{\theta} + J_1 \ddot{\alpha} - \frac{1}{2} J_1 \sin(2\alpha)\dot{\theta}^2 - G \sin\alpha = 0$$

3.8.1 What is used later

The controller chapters mainly use the following results from this derivation:

- Nonlinear upright relation:

$$\ddot{\alpha} = A \sin\alpha + \sin\alpha \cos\alpha\,\dot{\theta}^2 - B \cos\alpha\,\ddot{\theta}$$

- Linearized upright relation:

$$\ddot{\alpha} = 100.8\,\alpha - 1.952\,\ddot{\theta}$$

- Stepper conversion used in firmware and plots:

$$k_s = 4.444 \text{ step/deg}$$

So the main constants passed forward from this chapter are [A = 100.8, B = 1.952, k_s = 4.444 step/deg.]

These values match the constants used later in the firmware and controller chapters. They should be read as *model constants*, not as a guarantee that the real stepper-driven hardware follows the commanded acceleration exactly.

Chapter 4

Firmware Pipeline: Measurement and Estimation

4.1 Control loop timing

All controllers run in a fixed-period real-time loop on the Arduino Mega 2560. A *control tick* means one execution of this loop. In this project:

- the nominal tick period is $T_s = 5\,\text{ms}$ (that is, 200 Hz),
- each tick measures its actual duration using a microsecond timer,
- the measured dt is used for the main integrators and reference updates,
- the derivative filter coefficients are computed once from the nominal 5 ms period and are not recalculated from the measured per-tick dt,
- the CSV time-series stream is logged at about 50 Hz (decimated from the 200 Hz control tick).

Because timing jitter can affect derivative estimation and controller performance, the firmware monitors:

- minimum and maximum observed tick duration (dt_{min}, dt_{max}),
- the count of tick overruns beyond the nominal period, defined as $dt > T_{\text{ov}}$ with $T_{\text{ov}} = 6\,\text{ms}$.

These diagnostics are printed at 1 Hz and used as a sanity check when more computation is added, especially in nonlinear modes.

4.2 Derivative estimation

The control law needs $\dot{\alpha}$ and $\dot{\theta}$. A raw sample-to-sample difference is too noisy at 200 Hz, so the firmware uses a smoothed numerical derivative based on a first-order filtered differentiator and the bilinear (Tustin) transform [6, 7]:

$$H(s) = \frac{s}{1 + s/\omega_c}.$$

Applying the bilinear mapping $s \approx \frac{2}{T_s}\frac{1-z^{-1}}{1+z^{-1}}$ gives the discrete-time recursion

$$y[n] = b_0\,(x[n] - x[n-1]) - a_1\,y[n-1],$$

where $x[n]$ is the measured angle (in degrees), $y[n]$ is the filtered derivative (deg/s), and T_s is the nominal tick period:

$$b_0 = \frac{2\omega_c}{2 + \omega_c T_s}, \qquad a_1 = \frac{\omega_c T_s - 2}{2 + \omega_c T_s}.$$

The cutoff ω_c is tuned experimentally, and the coefficients (b_0, a_1) are computed using the nominal tick period T_s. For the firmware setting $\omega_c = 450\,\mathrm{rad/s}$ and $T_s = 5\,\mathrm{ms}$, the coefficients are

$$b_0 = 211.7647, \qquad a_1 = 0.0588.$$

Wrap handling and clamping. For the pendulum angle α, the pre-difference $(x[n] - x[n-1])$ is wrapped to $\pm 180°$ so the encoder wrap does not look like a false $\approx 360°$ jump. After filtering, both derivatives are clamped to conservative limits:

$$|\dot{\alpha}| \leq 300\,^\circ\,\mathrm{s}^{-1}, \qquad |\dot{\theta}| \leq 300\,^\circ\,\mathrm{s}^{-1}.$$

These limits keep single-sample spikes from dominating the controller.

4.3 Wrap safety and glitch rejection

The AS5600 sensors are read over I^{2}C via a multiplexer, so occasional mux or bus glitches can produce impossible one-sample jumps. The firmware applies three protections:

1. **Raw jump rejection:** if a single-tick jump exceeds 30° in either α or θ, the sample is discarded and the previous value is reused.

2. **Derivative-safe handling:** when a jump is rejected, the corresponding pre-difference is forced to zero so the derivative estimate does not see a large false spike.

3. **Reject accounting:** diagnostic counters record how often these protections are used.

All angle differences used in control and reference tracking use the wrap-safe operator described in Chapter 2.

Chapter 5

Firmware Pipeline: Actuation and Safety

5.1 Stepper actuation pipeline

The stepper driver is commanded in continuous-velocity mode. The controller measures the true base angle for feedback, but the motor command itself is still an open-loop speed command. All controllers compute a base acceleration command, which is then mapped to the motor direction convention and integrated to produce the commanded step rate:

$$u \equiv \ddot{\theta} \xrightarrow{\text{controller}} u_{\text{cmd}} \xrightarrow{\text{sign map}} u_{\text{cmd,motor}} \xrightarrow{\int} \dot{\theta}_{\text{cmd}}.$$

Here u_{cmd} is expressed as a step-acceleration command (units $\text{step}\,\text{s}^{-2}$), and $\dot{\theta}_{\text{cmd}}$ is the commanded step rate (units $\text{step}\,\text{s}^{-1}$). The integration is implemented as

$$\dot{\theta}_{\text{cmd}} \leftarrow \dot{\theta}_{\text{cmd}} + \left(\ddot{\theta}_{\text{cmd}} - \lambda_{\text{leak}}\dot{\theta}_{\text{cmd}}\right) T,$$

where λ_{leak} is optionally enabled only in the *linear* controller (Chapter 6) to suppress slow drift.

Low-speed stop deadband. Very small speed commands are suppressed to reduce buzz and dithering near zero motion. In the pinned dataset, the shared setting was

$$\texttt{speedStopHz} = 50\,\text{step}\,\text{s}^{-1}.$$

If the commanded speed magnitude stays below this threshold, the motor is stopped for that control tick.

Engage ramp. On transition into the active balancing state, the commanded acceleration is multiplied by an engage ramp to avoid impulsive start-up commands while derivatives and references are initializing. In firmware this ramp is a linear 100 ms rise:

$$u_{\text{cmd}} \leftarrow u_{\text{cmd}} \cdot \min\left(\frac{t - t_{\text{engage}}}{100\,\text{ms}}, 1\right).$$

Stop-before-reverse. To avoid direction chatter and mechanical jerk near zero, the firmware enforces a stop-before-reverse policy: if the desired direction changes sign while the motor is already running, the current tick commands a stop first and the reverse command can only be issued on a later control tick (≥ 5 ms later at 200 Hz).

5.2 Soft limits and failure modes

The firmware includes several safety layers to make experiments repeatable on a stepper-driven rig:

- **Soft base limits** near mechanical bounds,
- **stall detection** when the motor is commanded but the base does not move,
- **predictable disarm and idle behavior** so runs can be restarted quickly without power-cycling.

Soft limits. In addition to the hard base angle limit $|\theta| > 80°$ (Chapter 1), the implementation applies a soft limit margin $\Delta_\theta = 2°$. If the measured base angle is within this margin (that is, $|\theta| > 78°$) and the commanded velocity would push further outward, the velocity command is suppressed. This reduces the chance of pushing into the mechanical stop while the pendulum is still upright.

Stall detection (stall or missed-step events). In some runs, especially during aggressive disturbance recovery, the stepper can miss steps or fail to move despite a nonzero commanded velocity. The implementation checks this in three parts:

- **Threshold:** motion is expected only when $|\dot{\theta}_{\text{cmd}}| > \dot{\theta}_{\text{cmd,min}}$ with $\dot{\theta}_{\text{cmd,min}} = 500\,\text{step}\,\text{s}^{-1}$.
- **Persistence:** if the measured change stays below $|\Delta\theta| < \Delta\theta_{\text{min}}$ with $\Delta\theta_{\text{min}} = 0.10°$ for $N_{\text{stall}} = 30$ consecutive control ticks (150 ms at 200 Hz), the condition is treated as a stall or missed-motion event.
- **Diagnostic output:** the warning prints AS5600 base-sensor magnet diagnostics (status bits, AGC, magnitude) to help distinguish true motor or driver dropouts from sensor mounting problems.

Disarm and repeatability. When a hard limit is exceeded for $N_{\text{fall}} = 3$ consecutive control ticks (debounced, so 15 ms at 200 Hz), the firmware disables stepper outputs and returns to an idle state. The system remains ready to re-engage once the pendulum is returned upright, enabling rapid repeated trials without power-cycling.

5.3 Auto-trim and velocity leak

Two slow "bias killers" are implemented to improve repeatability under sensor offsets and small persistent disturbances:

- **Auto-trim of the upright reference:** when stable and not saturated, the upright reference is updated at $\lambda_{\text{trim}} = 0.20\,\text{s}^{-1}$, enabled only when $|\alpha| < \alpha_{\text{trim,win}} = 3°$ and $|\dot{\alpha}| < \dot{\alpha}_{\text{trim,win}} = 50\,°\,\text{s}^{-1}$. The accumulated trim offset is clamped to $|\alpha_{\text{trim}}| \leq \alpha_{\text{trim,max}} = 1.5°$ relative to the calibration reference.
- **Velocity leak (linear-only):** the acceleration integrator uses a leaky term with $\lambda_{\text{leak}} = 2.0\,\text{s}^{-1}$, enabled only when near upright ($|\alpha| < \alpha_{\text{leak,win}} = 5°$) and near the base reference ($|\theta_{\text{err}}| < \theta_{\text{leak,win}} = 20°$). This reduces slow random-walk drift due to static bias in the acceleration command.

The velocity leak is *disabled* in nonlinear modes. In SMC controllers, the acceleration command is derived directly from a nonlinear reaching law, so adding a leak would change the effective input channel.

Chapter 6

Linear Full-State Feedback Controller

6.1 Linearized plant for control design

This controller is the baseline controller used in the thesis. It uses all four states and tries to keep the pendulum upright while also keeping the arm near the reference angle.

The controller is designed about the upright equilibrium ($\alpha \approx 0$) using the acceleration-input formulation $u \equiv \ddot{\theta}$ from Chapter 5. Using the linearized model derived in Chapter 3, the upright dynamics can be written as

$$\mathbf{x} = \begin{bmatrix}\theta & \alpha & \dot{\theta} & \dot{\alpha}\end{bmatrix}^{\mathsf{T}} \equiv \begin{bmatrix}x_1 & x_2 & x_3 & x_4\end{bmatrix}^{\mathsf{T}}$$

and

$$\dot{x}_1 = x_3, \qquad \dot{x}_2 = x_4, \qquad \dot{x}_3 = u, \qquad \dot{x}_4 = A\,x_2 - B\,u. \tag{6.1}$$

For this rig,

$$A = 100.8, \qquad B = 1.952.$$

In state-space form,

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u, \qquad \mathbf{A} = \begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ 0 & 0 & 0 & 0\\ 0 & A & 0 & 0\end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix}0\\ 0\\ 1\\ -B\end{bmatrix}.$$

Linear state feedback is a common baseline for rotary inverted pendulum stabilization [8, 9].

6.2 Full-state feedback by pole placement

Choose the state-feedback law

$$u = -\mathbf{K}\mathbf{x}, \qquad \mathbf{K} = \begin{bmatrix}k_\theta & k_\alpha & k_{\dot{\theta}} & k_{\dot{\alpha}}\end{bmatrix}. \tag{6.2}$$

Substituting (6.2) into (6.1) gives the closed-loop matrix

$$\mathbf{A}_{\mathrm{cl}} = \mathbf{A} - \mathbf{B}\mathbf{K} = \begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ -k_\theta & -k_\alpha & -k_{\dot{\theta}} & -k_{\dot{\alpha}}\\ Bk_\theta & A + Bk_\alpha & Bk_{\dot{\theta}} & Bk_{\dot{\alpha}}\end{bmatrix}.$$

Eliminating θ from the scalar equations gives the fourth-order characteristic polynomial

$$s^4 + a_3 s^3 + a_2 s^2 + a_1 s + a_0 = 0, \tag{6.3}$$

with coefficients

$$a_3 = k_{\dot\theta} - Bk_{\dot\alpha}, \qquad a_2 = k_\theta - A - Bk_\alpha, \qquad a_1 = -Ak_{\dot\theta}, \qquad a_0 = -Ak_\theta. \tag{6.4}$$

This form is useful because the gains can be found directly by matching a desired polynomial [10].

6.2.1 Desired poles via two second-order factors

The desired closed-loop polynomial is chosen as a product of two second-order factors:

$$\left(s^2 + 2\zeta_1\omega_1 s + \omega_1^2\right)\left(s^2 + 2\zeta_2\omega_2 s + \omega_2^2\right) = s^4 + \tilde{a}_3 s^3 + \tilde{a}_2 s^2 + \tilde{a}_1 s + \tilde{a}_0, \tag{6.5}$$

where (ω_1, ζ_1) describe a faster pendulum mode and (ω_2, ζ_2) describe a slower base-centering mode. Expanding gives

$$\begin{aligned}
\tilde{a}_3 &= 2(\zeta_1\omega_1 + \zeta_2\omega_2),\\
\tilde{a}_2 &= \omega_1^2 + \omega_2^2 + 4\zeta_1\zeta_2\omega_1\omega_2,\\
\tilde{a}_1 &= 2(\zeta_1\omega_1\omega_2^2 + \zeta_2\omega_2\omega_1^2),\\
\tilde{a}_0 &= \omega_1^2\omega_2^2.
\end{aligned} \tag{6.6}$$

6.2.2 Closed-form gains

Setting $(a_0, a_1, a_2, a_3) = (\tilde{a}_0, \tilde{a}_1, \tilde{a}_2, \tilde{a}_3)$ and solving (6.4) yields

$$\begin{aligned}
k_\theta &= -\frac{\tilde{a}_0}{A}, & k_{\dot\theta} &= -\frac{\tilde{a}_1}{A},\\
k_{\dot\alpha} &= \frac{k_{\dot\theta} - \tilde{a}_3}{B}, & k_\alpha &= \frac{k_\theta - A - \tilde{a}_2}{B}.
\end{aligned} \tag{6.7}$$

6.2.3 Numerical example and unit conversion

Using $A = 100.8$, $B = 1.952$ and choosing a fast mode $(\omega_1, \zeta_1) = (15, 0.8)$ and a slow mode $(\omega_2, \zeta_2) = (1, 1)$, the desired coefficients are

$$\tilde{a}_3 = 26, \qquad \tilde{a}_2 = 274, \qquad \tilde{a}_1 = 474, \qquad \tilde{a}_0 = 225,$$

and the resulting gains from (6.7) are

$$\mathbf{K} = \begin{bmatrix} -2.232 & -193.15 & -4.702 & -15.729 \end{bmatrix}.$$

For implementation with degree states and stepper units, define the conversion k_s (units step $^{\circ -1}$; Chapter 2). The commanded acceleration in stepper units is

$$u_{\text{steps}} = k_s\, u_{\text{deg}}.$$

Therefore, the gain vector expressed in stepper-acceleration units is

$$\mathbf{K}_{\text{steps}} = k_s\, \mathbf{K}.$$

With $k_s = 4.444$, the example yields

$$\mathbf{K}_{\text{steps}} = \begin{bmatrix} -9.92 & -858.4 & -20.9 & -69.9 \end{bmatrix}.$$

This is a *design example*, not the final tuned parameter set used in the experiments.

6.3 Reference tracking and nudge behavior

Rather than regulating the base to $\theta = 0$ all the time, the implemented controller uses reference tracking:

$$\theta_{\text{err}} = \text{diff}(\theta, \theta_{\text{ref}}), \qquad \dot{\theta}_{\text{err}} = \dot{\theta} - \dot{\theta}_{\text{ref}},$$

with $\text{diff}(\cdot,\cdot)$ the wrap-safe difference operator from Chapter 2. The reference-tracking full-state feedback law is

$$u_{\text{cmd}} = u_{\text{ff}} + K_\theta\,\theta_{\text{err}} + K_\alpha\,\alpha + K_{\dot{\theta}}\,\dot{\theta}_{\text{err}} + K_{\dot{\alpha}}\,\dot{\alpha}, \tag{6.8}$$

with feedforward term $u_{\text{ff}} = \ddot{\theta}_{\text{ref}}$ provided by the reference generator.

Commanded base position while balancing (nudge mode)

The reference θ_{ref} is generated by a velocity- and acceleration-limited trapezoidal profile driven by a user-specified base angle target. This gives practical commanded base positioning while keeping the pendulum upright.

Because aggressive base motion can destabilize the pendulum, the reference generator includes a pause/resume mechanism. When instability is confirmed, the reference is handed once to the current measured θ and then held there until the pendulum becomes stable again. Once the pendulum returns to the stability window, the profile resumes.

In implementation, the move stability gate is defined as $|\alpha| < \alpha_{\text{move,win}}$ with $\alpha_{\text{move,win}} = 5^\circ$ and $|\dot{\alpha}| < \dot{\alpha}_{\text{move,win}}$ with $\dot{\alpha}_{\text{move,win}} = 80\,^\circ\,\text{s}^{-1}$. The gate is enabled only after a post-engage grace $T_g = 200\,\text{ms}$. Stability and instability are each debounced for 10 consecutive control ticks (50 ms at 200 Hz) before pausing or resuming the profile. For safety, user targets are also bounded by $|\theta_{\text{target}}| \leq 35^\circ$.

6.4 Implementation details (stepper-oriented)

Several practical additions are used to improve repeatability on real hardware:

- **Deadband on α:** $\alpha_{\text{db}} = 0.5^\circ$ reduces noise-chasing near upright.
- **Acceleration saturation and engage ramp:** the commanded acceleration is saturated to $u_{\max} = 20{,}000\,\text{step}\,\text{s}^{-2}$, the commanded speed is capped at $\dot{\theta}_{\max} = 15{,}000\,\text{step}\,\text{s}^{-1}$, and the command is multiplied by a linear engage ramp of duration $T_{\text{ramp}} = 100\,\text{ms}$ after controller engagement.
- **Low-speed stop deadband:** the motor is stopped for very small speed commands using `speedStopHz` set to $50\,\text{step}\,\text{s}^{-1}$.
- **Velocity leak (linear-only):** $\lambda_{\text{leak}} = 2.0\,\text{s}^{-1}$ is enabled only when $|\alpha| < \alpha_{\text{leak,win}} = 5^\circ$ and $|\theta_{\text{err}}| < \theta_{\text{leak,win}} = 20^\circ$ to suppress slow drift.
- **Auto-trim of the upright reference:** enabled only when $|\alpha| < \alpha_{\text{trim,win}} = 3^\circ$ and $|\dot{\alpha}| < \dot{\alpha}_{\text{trim,win}} = 50\,^\circ\,\text{s}^{-1}$; integrates at $\lambda_{\text{trim}} = 0.20\,\text{s}^{-1}$ and is clamped to $|\alpha_{\text{trim}}| \leq 1.5^\circ$ relative to the calibration reference.
- **Soft limits:** outward velocity commands are suppressed within a margin $\Delta_\theta = 2^\circ$ of the mechanical base limit.

6.5 Parameters used in experiments

The tuned firmware gains used in the pinned dataset are not the same as the pole-placement example above. The deployed gains differ because the implementation separates sign conventions from gain magnitudes, uses stronger base centering, and includes practical hardware tuning on the real rig.

Table 6.1: Linear controller gains used in the pinned experiments (stepper acceleration units).

Parameter	Value	Units	Meaning
K_θ	31.5	$\mathrm{step\,s^{-2}\,{}^\circ{}^{-1}}$	base or arm position gain
K_α	725.6	$\mathrm{step\,s^{-2}\,{}^\circ{}^{-1}}$	pendulum angle gain
$K_{\dot\theta}$	28.3	$\mathrm{step\,s^{-2}\,per\,(^\circ\,s^{-1})}$	base or arm rate gain
$K_{\dot\alpha}$	73.5	$\mathrm{step\,s^{-2}\,per\,(^\circ\,s^{-1})}$	pendulum rate gain

Derivative estimation uses the filtered differentiator described in Chapter 4 with cutoff $\omega_c = 450\,\mathrm{rad/s}$.

Observed behavior (HOLD/NUDGE/TAP). See Figures 10.2, 10.6, 10.8 and Tables in Chapter 10.

Chapter 7

Hybrid Sliding Mode Controller

7.1 Nonlinear upright dynamics (acceleration input)

The hybrid nonlinear controller uses the reduced nonlinear upright relation derived in Chapter 3. It is "exact" only within that reduced rigid-body model; it is not a full model of stalls, sensor glitches, or actuator saturation on the real rig. Written in acceleration-input form,

$$\ddot{\alpha} = A \sin\alpha \; + \; \sin\alpha \cos\alpha \, \dot{\theta}^2 \; - \; B \cos\alpha \, \ddot{\theta}. \tag{7.1}$$

For this rig,

$$A = 100.8, \qquad B = 1.952.$$

This model includes the gravity term $A \sin\alpha$ and the centrifugal coupling term $\sin\alpha \cos\alpha \, \dot{\theta}^2$. Sliding-mode variants are widely used for rotary inverted pendulum stabilization [11, 12, 13].

7.2 Sliding surface and motion on the surface

Define the pendulum-only sliding surface

$$s \; \equiv \; \dot{\alpha} + \lambda \, \alpha, \qquad \lambda > 0. \tag{7.2}$$

The surface is a combined error signal. If s goes to zero, the pendulum state is pushed toward upright. On the surface $s = 0$,

$$\dot{\alpha} = -\lambda\alpha \quad \Rightarrow \quad \alpha(t) = \alpha_0 e^{-\lambda t},$$

so λ sets the convergence speed in the ideal sliding phase.

7.3 Surface dynamics and equivalent control

Differentiating (7.2) gives $\dot{s} = \ddot{\alpha} + \lambda\dot{\alpha}$. Substituting (7.1) yields

$$\dot{s} = \Big(A \sin\alpha + \sin\alpha \cos\alpha \, \dot{\theta}^2 + \lambda\dot{\alpha} \Big) - \; B \cos\alpha \, \ddot{\theta}. \tag{7.3}$$

Define the drift term

$$f(\alpha, \dot{\alpha}, \dot{\theta}) \equiv A \sin\alpha + \sin\alpha \cos\alpha \, \dot{\theta}^2 + \lambda\dot{\alpha},$$

so that (7.3) becomes $\dot{s} = f - B \cos\alpha \, \ddot{\theta}$. The equivalent control, which keeps $\dot{s} = 0$, is

$$\ddot{\theta}_{\text{eq}} = \frac{f}{B \cos\alpha}. \tag{7.4}$$

7.4 Reaching law and complete control law

An ideal reaching law uses a discontinuous sign term, $\dot{s} = -K\,\mathrm{sgn}(s)$, but that is too harsh on real hardware. The implemented controller therefore uses a boundary layer via a saturation function [14, 15, 16]:

$$\dot{s} = -K\,\mathrm{sat}\left(\frac{s}{\phi}\right), \qquad K > 0,\ \phi > 0, \tag{7.5}$$

where

$$\mathrm{sat}(x) = \begin{cases} +1, & x > 1 \\ x, & |x| \le 1 \\ -1, & x < -1. \end{cases}$$

Substituting (7.5) into $\dot{s} = f - B\cos\alpha\,\ddot{\theta}$ gives

$$\ddot{\theta} = \frac{f(\alpha, \dot{\alpha}, \dot{\theta}) + K\,\mathrm{sat}\left(\frac{s}{\phi}\right)}{B\cos\alpha}. \tag{7.6}$$

Outside the boundary layer ($|s| > \phi$), the controller pulls strongly toward the surface. Inside the boundary layer, the pull becomes smoother and chattering is reduced.

7.5 Lyapunov stability analysis (ideal reaching law)

The reaching law is designed to drive the sliding variable s in (7.2) toward zero. A standard Lyapunov choice is

$$V(s) = \frac{1}{2}s^2, \qquad V \ge 0.$$

Using the boundary-layer reaching law (7.5) [14, 15, 16],

$$\dot{V} = s\dot{s} = -K\,s\,\mathrm{sat}\left(\frac{s}{\phi}\right) \le 0.$$

This means V does not increase, so the ideal analytic law pushes s toward the sliding surface.

- If $|s| > \phi$, then $\mathrm{sat}(s/\phi) = \mathrm{sgn}(s)$ and $\dot{V} = -K|s| < 0$.
- If $|s| \le \phi$, then $\mathrm{sat}(s/\phi) = s/\phi$ and $\dot{V} = -(K/\phi)s^2 \le 0$.

Theory versus firmware. The Lyapunov argument above applies to the analytic law before practical limits are added. The deployed firmware also includes actuator saturation, a speed deadband, engage ramping, sign handling, cosine protection, upright-only aborts, alpha-dot abort logic, and a drift-reduction base assist. These additions are necessary on the real stepper-driven rig, but they mean the ideal reaching claim should be read as a design guide, not as an unconditional hardware guarantee.

7.6 Stepper-oriented implementation constraints

The control law (7.6) is intended for upright stabilization only. In practice, the firmware adds several guards:

- **Upright-only window:** abort if $|\alpha| > 25°$.
- **Cosine floor:** replace $\cos\alpha$ by a protected $\cos_{\mathrm{safe}} = \mathrm{sgn}(\cos\alpha)\max(|\cos\alpha|, 0.2)$.

- **Rate protection:** abort if $|\dot{\alpha}| > 250\,^\circ\,\mathrm{s}^{-1}$ for the required debounce window.
- **Acceleration cap:** clamp the base acceleration command to $u_{\max} = 20{,}000\,\mathrm{step\,s}^{-2}$.
- **Command sign:** the firmware includes `smcSign` so the physical actuation direction can be corrected if needed without changing the printed equations.

The nonlinear computations use $\sin(\alpha)$ and $\cos(\alpha)$ with α interpreted in radians; angles are recorded and plotted in degrees for readability.

7.7 Base-centering assist term (why this controller is "hybrid")

The surface (7.2) stabilizes the pendulum, but by itself it does not directly regulate the base angle. On the real rig, small biases can make the arm drift slowly even while the pendulum stays upright. The firmware therefore adds a practical base-centering assist:

$$\ddot{\theta}_{\mathrm{base}} = \ddot{\theta}_{\mathrm{ref}} + K_\theta\,\theta_{\mathrm{err}} + K_{\dot{\theta}}\,\dot{\theta}_{\mathrm{err}}, \tag{7.7}$$

where

$$\theta_{\mathrm{err}} = \mathrm{diff}(\theta, \theta_{\mathrm{ref}}), \qquad \dot{\theta}_{\mathrm{err}} = \dot{\theta} - \dot{\theta}_{\mathrm{ref}}.$$

This assist is a practical fix for drift. It is not part of the classical pendulum-only sliding surface.

The total command in stepper acceleration units is

$$u_{\mathrm{cmd}} \;=\; u_{\mathrm{smc}} + \gamma\,u_{\mathrm{base}},$$

where u_{smc} is (7.6) and u_{base} corresponds to (7.7) expressed in the same units. In implementation, the centering assist is scaled by γ (default $\gamma = 1.0$, range $0\ldots2$) and gated by an upright and still window: it is enabled only after a post-engage grace of 200 ms and when $|\alpha| < 5^\circ$ and $|\dot{\alpha}| < 150\,^\circ\,\mathrm{s}^{-1}$.

7.8 Parameters used in experiments

For the pinned dataset, the hybrid SMC used: The base-centering gains $(K_\theta, K_{\dot{\theta}})$ are shared

Table 7.1: Hybrid sliding-mode controller parameters used in experiments.

Parameter	Value	Meaning
λ	$15\ \mathrm{s}^{-1}$	sliding surface slope in (7.2)
K	$800\ ^\circ\,\mathrm{s}^{-2}$	reaching gain in (7.5)
ϕ	$50\ ^\circ\,\mathrm{s}^{-1}$	boundary layer thickness
γ	1.0	base-centering scale
`smcSign`	+1	actuation-direction sign used by firmware
$\lvert\alpha\rvert$ validity	25°	upright-only validity window
$\lvert\dot{\alpha}\rvert$ abort	$250\,^\circ\,\mathrm{s}^{-1}$	extreme-rate protection (debounced)

with the linear controller and are gated so they do not fight recovery during large excursions.

Observed behavior (HOLD/NUDGE/TAP). See Figures 10.2, 10.8 and Tables in Chapter 10.

Chapter 8

Full-Surface Sliding Mode Controller (Four-State Surface)

8.1 Base tracking errors

The base terms in SMC4 are written in tracking-error form so the same controller can cover both hold-at-center ($\theta_{\text{ref}} = \text{const}$) and commanded moves (nudge mode, time-varying θ_{ref}). Define

$$\theta_{\text{err}} \equiv \text{diff}(\theta, \theta_{\text{ref}}), \qquad \dot{\theta}_{\text{err}} \equiv \dot{\theta} - \dot{\theta}_{\text{ref}},$$

with the wrap-safe operator $\text{diff}(\cdot,\cdot)$ from Chapter 2. The reference acceleration $\ddot{\theta}_{\text{ref}}$ appears as a feedforward term in the surface dynamics.

8.2 Full-state sliding surface

SMC4 extends the pendulum-only sliding surface by adding base-tracking error terms:

$$s \equiv \dot{\alpha} + \lambda_\alpha \alpha + k\Big(\dot{\theta}_{\text{err}} + \lambda_\theta \theta_{\text{err}}\Big), \tag{8.1}$$

with design parameters $\lambda_\alpha > 0$, $\lambda_\theta > 0$, and coupling weight $k \geq 0$. The combined base term

$$\dot{\theta}_{\text{err}} + \lambda_\theta \theta_{\text{err}}$$

is just a single signal that says how far the base is from the desired tracking behavior. If $k = 0$, (8.1) reduces to the pendulum-only surface used in Chapter 7.

8.3 Surface dynamics and control effectiveness

Differentiating (8.1) gives

$$\dot{s} = \ddot{\alpha} + \lambda_\alpha \dot{\alpha} + k\ddot{\theta}_{\text{err}} + k\lambda_\theta \dot{\theta}_{\text{err}}. \tag{8.2}$$

Using $\ddot{\theta}_{\text{err}} = \ddot{\theta} - \ddot{\theta}_{\text{ref}}$ and substituting the nonlinear relation (7.1) yields

$$\dot{s} = \underbrace{\Big(A \sin\alpha + \sin\alpha \cos\alpha\, \dot{\theta}^2\Big)}_{\triangleq f_0(\alpha,\dot{\theta})} + \lambda_\alpha \dot{\alpha} \;+\; k\lambda_\theta \dot{\theta}_{\text{err}} \;-\; k\ddot{\theta}_{\text{ref}} \;+\; (k - B\cos\alpha)\ddot{\theta}. \tag{8.3}$$

The important point is the factor $(B\cos\alpha - k)$. If k gets too close to $B\cos\alpha$, the controller loses authority and asks for very large accelerations. That is why the firmware protects the denominator.

8.4 Reaching law and complete control law

As in Chapter 7, use the boundary-layer reaching law

$$\dot{s} = -K\,\mathrm{sat}\left(\frac{s}{\phi}\right), \qquad K > 0,\ \phi > 0.$$

Substituting into (8.3) gives the analytic control law

$$\boxed{\ddot{\theta} = \frac{f_0(\alpha,\dot{\theta}) + \lambda_\alpha\dot{\alpha} + k\lambda_\theta\dot{\theta}_{\mathrm{err}} - k\ddot{\theta}_{\mathrm{ref}} + K\,\mathrm{sat}\left(\frac{s}{\phi}\right)}{B\cos\alpha - k}} \tag{8.4}$$

with s given by (8.1). Equation (8.4) is the *analytic form*. The firmware does not use k directly at runtime; it uses an effective coupling k_{Eff} after ramping and denominator protection.

8.5 Lyapunov stability analysis (ideal reaching law)

As in the hybrid controller, the reaching law is designed to drive the sliding variable s in (8.1) toward zero. Choose the Lyapunov candidate

$$V(s) = \frac{1}{2}s^2, \qquad V \geq 0.$$

With the boundary-layer reaching law $\dot{s} = -K\,\mathrm{sat}(s/\phi)$ [14, 15, 16],

$$\dot{V} = s\dot{s} = -K\,s\,\mathrm{sat}\left(\frac{s}{\phi}\right) \leq 0.$$

Therefore V never increases and the ideal analytic law drives s toward the sliding surface.

Theory versus firmware. As in Chapter 7, this Lyapunov argument applies to the analytic law before practical limits are added. The real firmware also includes k ramping, denominator-margin protection, clamped θ_{err}, clamped $\dot{\theta}_{\mathrm{err}}$, clamped internal $\dot{\alpha}$, extra acceleration and speed caps, sign handling, and a gated base assist. These protections are necessary for the stepper-driven platform, but they mean the ideal reaching result is not a full hardware guarantee.

8.6 Implementation safeguards and stepper-friendly shaping

Compared to hybrid SMC, SMC4 is more sensitive because the coupling term appears both in the surface and in the denominator. The firmware therefore adds several safeguards:

- **Upright-only and cosine floor:** enforce $|\alpha| < 25°$ and use a protected $\cos_{\mathrm{safe}} = \mathrm{sgn}(\cos\alpha)\max(|\cos\alpha|, 0.2)$.
- **Limit coupling:** enforce $k \leq k_{\max} = 1.20$.
- **Denominator margin:** enforce $|B\cos_{\mathrm{safe}} - k_{\mathrm{Eff}}| \geq \mathrm{den}_{\min} = 0.60$.
- **Smooth engage:** ramp the coupling from zero to its requested value over 200 ms. The diagnostic fields `kRamp`, `kEff`, and `den` printed by the firmware show this protection in action.
- **Practical clamps:** clamp $|\theta_{\mathrm{err}}| \leq 78°$, clamp $|\dot{\theta}_{\mathrm{err}}| \leq 200\,°\,\mathrm{s}^{-1}$, clamp the internal $\dot{\alpha}$ used in the computation to $|\dot{\alpha}| \leq 200\,°\,\mathrm{s}^{-1}$, and apply tighter caps $u_{\max,\mathrm{SMC4}} = 12{,}000\,\mathrm{step\,s}^{-2}$ and $\dot{\theta}_{\max,\mathrm{SMC4}} = 2{,}000\,\mathrm{step\,s}^{-1}$.

8.6.1 Gated base-centering assist (implemented)

Although the surface (8.1) already includes base error terms, the firmware also adds a small gated base-centering assist to reduce slow drift once the pendulum is already near upright. This assist uses the same practical centering idea introduced in Chapter 7. It is scaled by γ (default $\gamma = 1.0$) and enabled only after a 200 ms grace period and when $|\alpha| < 5^\circ$ and $|\dot{\alpha}| < 250\,^\circ\,\mathrm{s}^{-1}$.

8.7 Parameters used in experiments

For the pinned dataset, the full-surface controller used: As with hybrid SMC, upright-only

Table 8.1: Full-surface sliding-mode controller parameters used in experiments.

Parameter	Value	Meaning
λ_α	$15\ \mathrm{s}^{-1}$	pendulum surface slope
K	$800\ ^\circ\,\mathrm{s}^{-2}$	reaching gain
ϕ	$50\ ^\circ\,\mathrm{s}^{-1}$	boundary layer thickness
k	0.50	requested base coupling weight in (8.1)
λ_θ	$2.0\ \mathrm{s}^{-1}$	base error shaping in (8.1)
γ	1.0	gated base-assist scale used in firmware
$k_{\max}$	1.20	hard coupling limit
$\mathrm{den}_{\min}$	0.60	minimum denominator margin
coupling ramp	200 ms	ramp-in duration after engage

validity and extreme-rate protections are enabled, and the final commanded acceleration is clamped before it is applied to the stepper.

Observed behavior (HOLD/NUDGE/TAP). See Figures 10.2, 10.8 and Tables in Chapter 10.

Chapter 9

Experiments and Data Processing

9.1 Dataset definition

All results in this report are generated from one fixed dataset of experimental runs selected through the manifest `experiments/manifest_thesis_20260209.json`. This makes the analysis repeatable: the same report can be rebuilt from the same seven pinned runs.

Report-facing run identifiers

For readability, the internal run labels used by the analysis pipeline are mapped to report-facing identifiers:

- Hold-at-center: **H1–H3**
- Commanded reference tracking (nudge): **N1**
- Finger tap disturbance: **T1–T3**

Each condition is represented by one pinned run only, so later comparisons are qualitative and metric-based rather than statistical. Table 9.1 lists the run identifiers used for this report.

Table 9.1: Dataset used for this report (manifest sessions).

ID	Controller	Type	Run ID
H1	Linear	Hold	20260209_095631
H2	Hybrid SMC	Hold	20260209_095908
H3	Full-surface SMC	Hold	20260209_121635
N1	Linear	Nudge	20260209_114605
T1	Linear	Tap	20260209_114857
T2	Hybrid SMC	Tap	20260209_114953
T3	Full-surface SMC	Tap	20260209_121803

9.2 Controller parameters used in experiments

For completeness, Table 9.2 summarizes the numeric controller and pipeline parameters used to generate the results in Chapter 10. These values correspond to the fixed configurations used during the pinned dataset runs H1–H3, N1, and T1–T3.

Table 9.2: Controller and pipeline parameters used in the pinned experimental dataset.

Category	Parameter	Value	Units	Notes
Shared	Stepper conversion k_s	4.444	$\mathrm{step}\,^{\circ -1}$	used to convert degrees to step units
Shared	Control tick	200	Hz	fixed-period loop
Shared	CSV logging rate	50	Hz	decimated from control tick
Shared	Nominal tick period T_s	5	ms	target loop period
Shared	Overrun threshold T_{ov}	6	ms	counted if $dt > T_{\mathrm{ov}}$
Shared	Base angle limit	80	$^\circ$	hard disarm beyond this bound
Shared	Pendulum angle limit	30	$^\circ$	hard disarm beyond this bound
Shared	Hard limit debounce N_{fall}	3	ticks	15 ms at 200 Hz
Shared	Soft limit margin Δ_θ	2	$^\circ$	clamps outward motion for $\lvert\theta\rvert > 78^\circ$
Shared	Engage ramp duration T_{ramp}	100	ms	linear ramp applied after controller engagement
Shared	Acceleration cap $u_{\max}$	20000	$\mathrm{step\,s^{-2}}$	actuator shaping
Shared	Speed cap $\dot{\theta}_{\max}$	15000	$\mathrm{step\,s^{-1}}$	actuator shaping
Shared	Speed stop deadband `speedStopHz`	50	$\mathrm{step\,s^{-1}}$	motor stopped for smaller speed commands
Shared	Stall detect min cmd speed $\dot{\theta}_{\mathrm{cmd,min}}$	500	$\mathrm{step\,s^{-1}}$	expect-motion threshold
Shared	Stall detect min motion $\Delta\theta_{\min}$	0.10	$^\circ$	per control tick
Shared	Stall detect persistence N_{stall}	30	ticks	150 ms at 200 Hz
Shared (nonlinear)	Upright-only validity $\lvert\alpha\rvert$	25	$^\circ$	abort to enforce upright-only operation
Shared (nonlinear)	Extreme-rate abort $\lvert\dot{\alpha}\rvert$	250	$^\circ\,\mathrm{s}^{-1}$	debounced, prevents rail commands
Shared (nonlinear)	Extreme-rate abort debounce $N_{\dot{\alpha}}$	3	ticks	consecutive ticks above threshold
Shared (nonlinear)	Extreme-rate abort grace T_g	200	ms	post-engage grace
Shared (nonlinear)	Extreme-rate abort min $\lvert\alpha\rvert$ $\alpha_{\min}$	5	$^\circ$	min-angle gate to avoid noise trips
Shared (nonlinear)	Cosine floor $\cos_{\min}$	0.2	–	used in $\cos_{\mathrm{safe}}$
Linear	K_θ	31.5	$\mathrm{step\,s^{-2}}\,^{\circ -1}$	base or arm position gain
Linear	K_α	725.6	$\mathrm{step\,s^{-2}}\,^{\circ -1}$	pendulum angle gain
Linear	$K_{\dot{\theta}}$	28.3	$\mathrm{step\,s^{-2}}$ per $(^\circ\,\mathrm{s}^{-1})$	base or arm rate gain
Linear	$K_{\dot{\alpha}}$	73.5	$\mathrm{step\,s^{-2}}$ per $(^\circ\,\mathrm{s}^{-1})$	pendulum rate gain
Linear	Derivative filter cutoff ω_c	450	rad/s	bilinear filtered differentiator
Linear	α deadband α_{db}	0.5	$^\circ$	reduces noise chasing near upright
Linear	Velocity leak coefficient λ_{leak}	2.0	s^{-1}	enabled only near upright and center
Linear	Velocity leak window $\lvert\alpha\rvert$ $\alpha_{\mathrm{leak,win}}$	5	$^\circ$	enables leak only near upright
Linear	Velocity leak window $\lvert\theta_{\mathrm{err}}\rvert$ $\theta_{\mathrm{leak,win}}$	20	$^\circ$	enables leak only near center
Linear	Upright reference trim rate	0.20	s^{-1}	slow bias compensation

(Continued on next page)

Category	Parameter	Value	Units	Notes
Linear	Auto-trim clamp $\alpha_{\text{trim,max}}$	1.5	$^\circ$	relative to calibration reference
Linear	Auto-trim window $\lvert\alpha\rvert$ $\alpha_{\text{trim,win}}$	3	$^\circ$	enables trim only near upright
Linear	Auto-trim window $\lvert\dot{\alpha}\rvert$ $\dot{\alpha}_{\text{trim,win}}$	50	$^\circ\,\text{s}^{-1}$	enables trim only when still
Hybrid SMC	Surface slope λ	15	s^{-1}	$s = \dot{\alpha} + \lambda\alpha$
Hybrid SMC	Reaching gain K	800	$^\circ\,\text{s}^{-2}$	boundary-layer reaching law
Hybrid SMC	Boundary layer ϕ	50	$^\circ\,\text{s}^{-1}$	reduces chattering
Hybrid SMC	Base assist scale γ	1.0	–	gated base-centering assist
Hybrid SMC	`smcSign`	+1	–	firmware sign correction for actuation direction
Hybrid SMC	Base assist gate $\lvert\alpha\rvert$ α_γ	5	$^\circ$	enables centering assist only near upright
Hybrid SMC	Base assist gate $\lvert\dot{\alpha}\rvert$ $\dot{\alpha}_\gamma$	150	$^\circ\,\text{s}^{-1}$	enables centering assist only when still
Full-surface SMC	Pendulum slope λ_α	15	s^{-1}	part of four-state surface
Full-surface SMC	Base slope λ_θ	2.0	s^{-1}	shapes base tracking term
Full-surface SMC	Coupling weight k	0.50	–	requested coupling in the analytic surface
Full-surface SMC	Reaching gain K	800	$^\circ\,\text{s}^{-2}$	boundary-layer reaching law
Full-surface SMC	Boundary layer ϕ	50	$^\circ\,\text{s}^{-1}$	reduces chattering
Full-surface SMC	Assist scale γ	1.0	–	gated base-centering assist
Full-surface SMC	Coupling hard limit $k_{\max}$	1.20	–	prevents ill-conditioning
Full-surface SMC	Denominator margin $\text{den}_{\min}$	0.60	–	enforces denominator protection
Full-surface SMC	Coupling ramp duration	200	ms	ramp-in after engage
Full-surface SMC	Internal $\dot{\alpha}$ clamp $\dot{\alpha}_{\text{ctrl,max}}$	200	$^\circ\,\text{s}^{-1}$	limits spikes in the internal computation
Full-surface SMC	Error clamp $\lvert\theta_{\text{err}}\rvert$ $\theta_{\text{err,max}}$	78	$^\circ$	explicit base error clamping
Full-surface SMC	Error clamp $\lvert\dot{\theta}_{\text{err}}\rvert$ $\dot{\theta}_{\text{err,max}}$	200	$^\circ\,\text{s}^{-1}$	explicit base error clamping
Full-surface SMC	Base term clamp $\theta_{\text{term,max}}$	300	$^\circ\,\text{s}^{-1}$	clamps the combined base term
Full-surface SMC	Extra accel cap $u_{\text{max,SMC4}}$	12000	$\text{step}\,\text{s}^{-2}$	tighter shaping than the global cap
Full-surface SMC	Extra speed cap $\dot{\theta}_{\text{max,SMC4}}$	2000	$\text{step}\,\text{s}^{-1}$	tighter shaping than the global cap
Full-surface SMC	Base assist gate $\lvert\dot{\alpha}\rvert$ $\dot{\alpha}_{\gamma,\text{SMC4}}$	250	$^\circ\,\text{s}^{-1}$	enables assist in the calmer region
Reference tracking	Target limit $\lvert\theta_{\text{target}}\rvert$ $\theta_{\text{target,max}}$	35	$^\circ$	bounds user-specified targets
Reference tracking	Max reference speed	12	$^\circ\,\text{s}^{-1}$	trapezoidal profile limit

(Continued on next page)

Category	Parameter	Value	Units	Notes
Reference tracking	Max reference acceleration	60	$^\circ\,\mathrm{s}^{-2}$	trapezoidal profile limit
Reference tracking	Stability gate $\lvert\alpha\rvert$ $\alpha_{\mathrm{move,win}}$	5	$^\circ$	required to keep the moving profile active
Reference tracking	Stability gate $\lvert\dot{\alpha}\rvert$ $\dot{\alpha}_{\mathrm{move,win}}$	80	$^\circ\,\mathrm{s}^{-1}$	required to keep the moving profile active
Reference tracking	Engage grace T_g	200	ms	before enabling pause or resume gating
Reference tracking	Pause/resume debounce $N_{\mathrm{stable}} = N_{\mathrm{unstable}}$	10	ticks	consecutive ticks required

9.3 Logging artifacts

Each logger session produces:

- a device-time time-series log (about 50 Hz),
- a host-time event log (status and diagnostic messages).

Within a session, each controller engagement followed by a disarm defines one *trial*.

The hold and nudge runs are straightforward to interpret from the recorded trajectories. The tap runs are less direct: they do not contain explicit disturbance markers, so the dataset supports qualitative disturbance-recovery comparisons but not precise impulse-response timing metrics.

9.4 Event-to-CSV alignment and limitations

The event log timestamps are host wall-clock time, while the time-series log uses device time. Therefore, naive alignment can introduce 100 ms to 500 ms shifts and corrupt any time-window metrics.

The analysis pipeline aligns host events to device time by matching sparse status snapshots to nearby device-time samples using a multi-signal error metric (for example α, $\dot{\alpha}$, θ, $\dot{\theta}$, acceleration, and velocity) under a monotonicity constraint. A linear host-to-device mapping is then fit and residuals are reported. If alignment quality is unreliable, the pipeline avoids precise event-time annotations and instead reports metrics computed directly from device-time trajectories.

9.5 Metric definitions

All metrics are defined once here and reused in later tables:

- **RMS and max:** $\mathrm{RMS}(\lvert\alpha\rvert)$, $\max\lvert\alpha\rvert$, $\max\lvert\theta\rvert$, and related quantities over the trial duration.
- **Drift slope:** slope of $\theta(t)$ over the trial (least-squares fit), reported in $^\circ\,\mathrm{s}^{-1}$.
- **Effort:** RMS and max of the commanded base acceleration u in $\mathrm{step}\,\mathrm{s}^{-2}$.
- **Saturation and clamp percentages:** fraction of samples where acceleration saturation is active and fraction where the base is near the mechanical limit.

Chapter 10

Results and Discussion

This chapter compares the pinned thesis dataset using the figures and tables generated by the analysis pipeline described in Chapter 9. Because each condition is represented by one pinned run only, all conclusions in this chapter are qualitative and metric-based.

10.1 How to read this chapter

The dataset used in this report consists of seven pinned representative trials: H1–H3 (hold), N1 (nudge), and T1–T3 (tap). The main body uses shared comparison plots so the three controllers can be read on the same scale. Supporting diagnostics are shown afterward for readers who want more detail.

10.2 Trials overview

Table 10.1 lists the seven pinned trials used in this report.

Table 10.1: Trials overview (report-facing identifiers H/N/T).

Trial	Mode	Type	Outcome	Duration (s)	α RMS (°)	max $\lvert\alpha\rvert$ (°)	max $\lvert\theta\rvert$ (°)	θ drift (°s^{-1})	acc RMS (step s^{-2})
H1	Linear	Hold	success	70.039	0.513	6.76	12.39	0.0	1354.846
H2	Hybrid SMC	Hold	success	99.005	0.826	4.15	8.52	0.0	1465.582
H3	Full-surface SMC	Hold	success	57.085	1.61	7.88	13.0	-0.142	3364.957
N1	Linear	Nudge	success	50.337	0.861	3.85	29.61	-0.077	2013.631
T1	Linear	Tap	success	31.293	0.864	5.92	37.7	0.0	1695.658
T2	Hybrid SMC	Tap	failed_fallen	27.877	1.847	24.87	54.93	-0.155	1493.067
T3	Full-surface SMC	Tap	failed_fallen	31.466	2.524	22.61	38.23	0.0	3843.042

10.3 Upright hold-at-center (H1–H3)

Objective. Each controller is engaged with $\theta_{\text{ref}} \equiv 0$ and asked to balance upright without drifting toward the mechanical base limits.
Key takeaways.

- The linear controller gives the lowest RMS upright error in the pinned hold runs.
- The hybrid SMC run gives the smallest peak $|\alpha|$ and the smallest peak $|\theta|$ in this fixed dataset.
- The full-surface SMC run also balances upright, but it uses much higher effort and shows noticeable centering drift in its representative trial.

Summary metrics.

Table 10.2: Summary metrics by controller mode (hold experiments).

Mode	Duration (s)	α RMS (°)	max $\lvert\alpha\rvert$ (°)	max $\lvert\theta\rvert$ (°)	θ drift (° s^{-1})	acc RMS (step s^{-2})	max $\lvert acc\rvert$ (step s^{-2})
Linear	70.039	0.513	6.76	12.39	0.0	1354.846	13640.0
Hybrid SMC	99.005	0.826	4.15	8.52	0.0	1465.582	5027.0
Full-surface SMC	57.085	1.61	7.88	13.0	-0.142	3364.957	12000.0

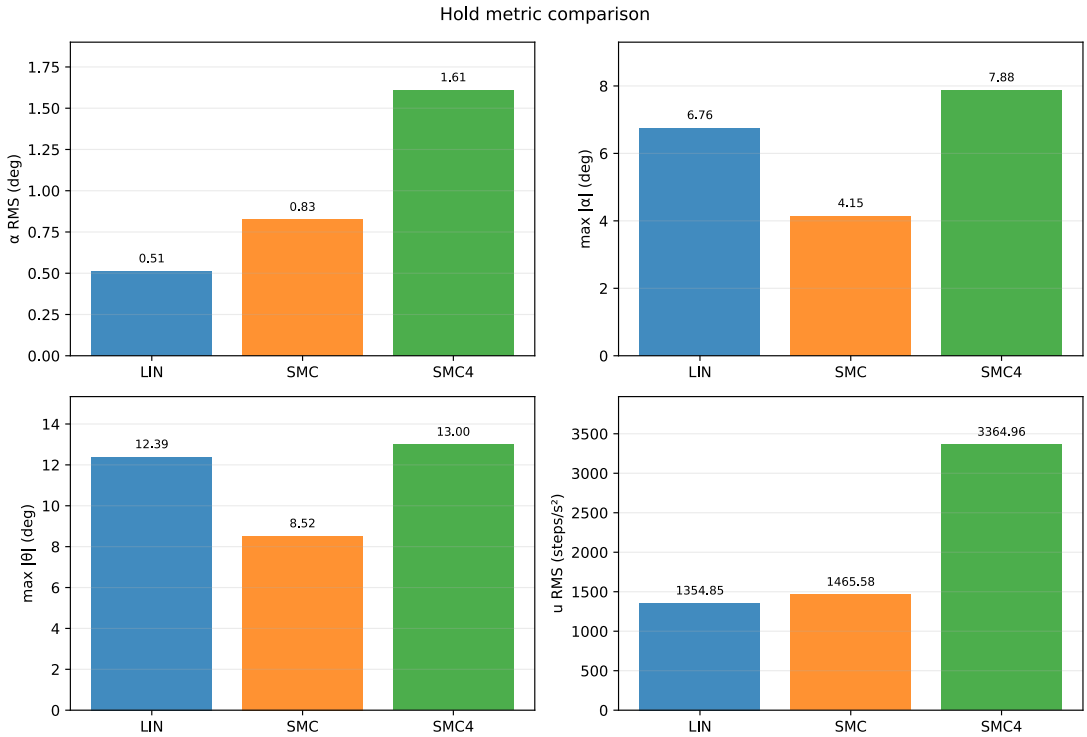


Figure 10.1: Compact comparison of the pinned hold metrics used in the discussion: α RMS, max $|\alpha|$, max $|\theta|$, and acceleration RMS.

Interpretation. The pinned hold results show a clear tradeoff on this rig. The linear controller is the calmest around upright, the hybrid SMC suppresses peak excursions best in this benign hold case, and the full-surface SMC pays a clear effort cost for its more aggressive nonlinear action.

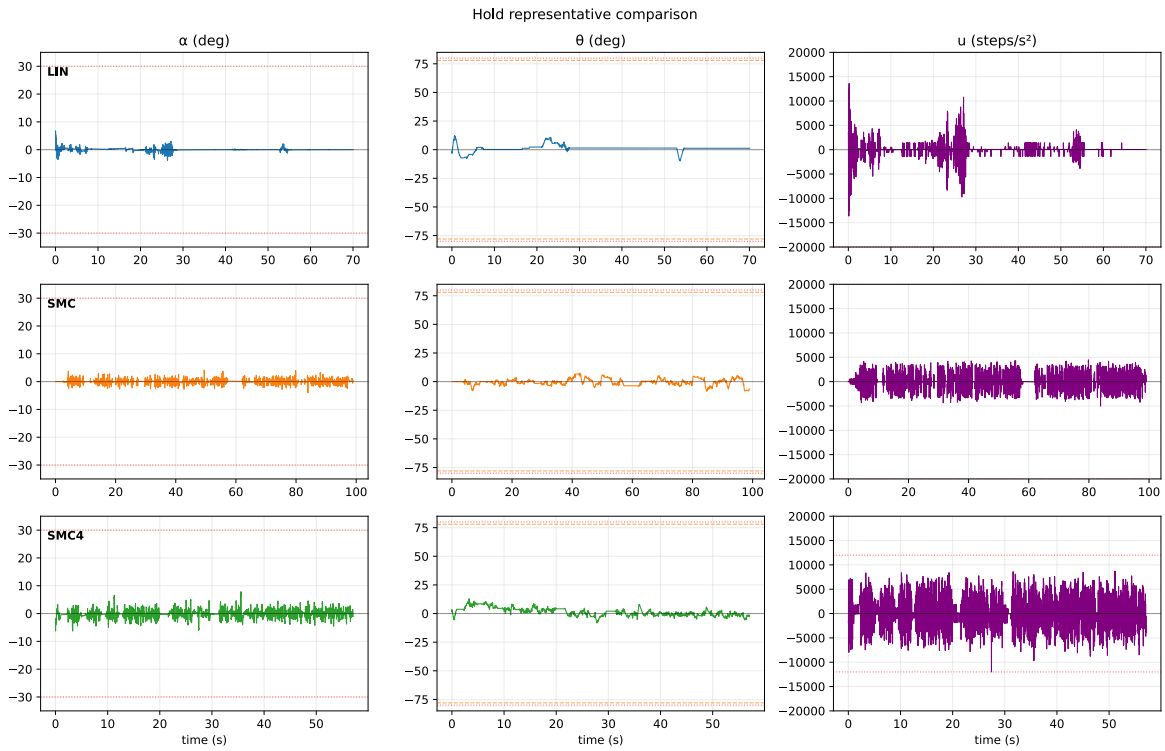
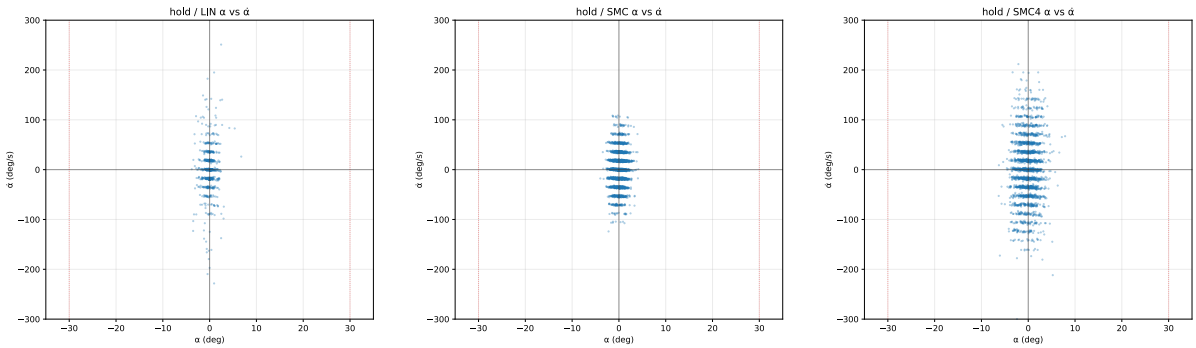


Figure 10.2: Common-scale comparison panel for the pinned hold runs. Each row is one controller, and each column shows $\alpha(t)$, $\theta(t)$, or commanded acceleration. Shared scales make the controller differences easier to compare directly.

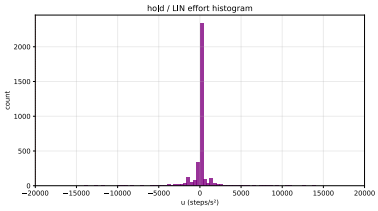
Supporting diagnostics

The following figures do not change the main conclusion, but they make the comparison easier to inspect in detail.

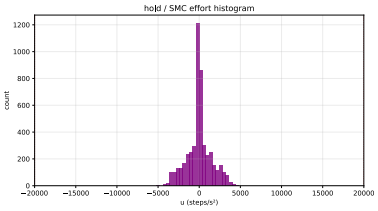


(a) Linear (b) Hybrid SMC (c) Full-surface SMC

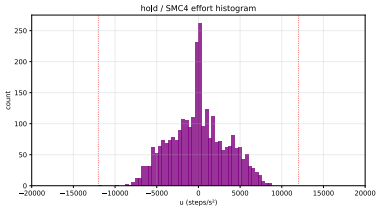
Figure 10.3: Phase portraits for the pinned hold runs. These plots show how tightly each controller keeps the pendulum state around upright.



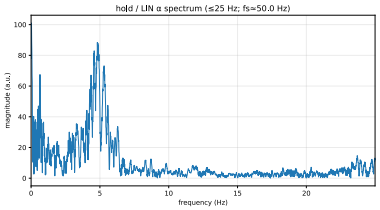
(a) Linear effort



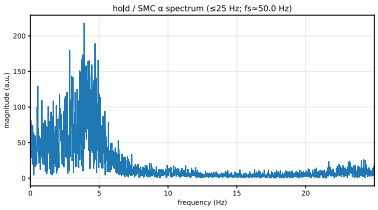
(b) Hybrid SMC effort



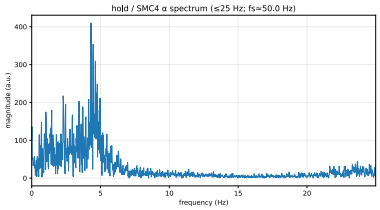
(c) Full-surface SMC effort



(d) Linear spectrum

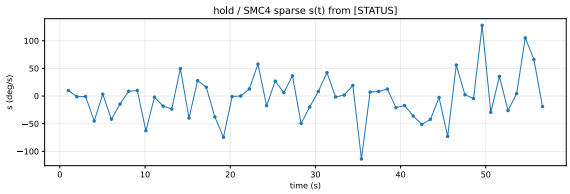


(e) Hybrid SMC spectrum

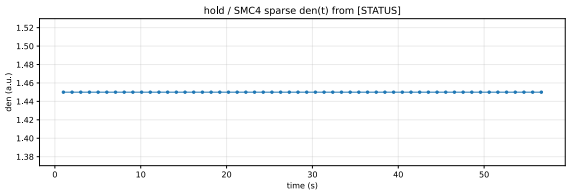


(f) Full-surface SMC spectrum

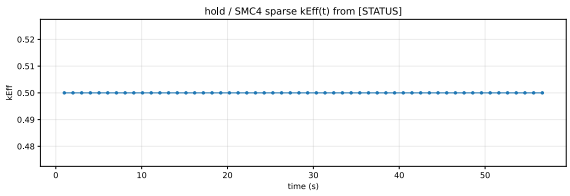
Figure 10.4: Top row: effort histograms for the pinned hold runs. Bottom row: low-frequency α spectra. The spectra are low-frequency only (≤ 25 Hz) because the thesis logs are recorded at about 50 Hz.



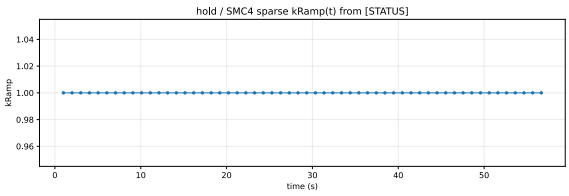
(a) Sliding surface $s(t)$



(b) Protected denominator



(c) Effective coupling k_{Eff}



(d) Coupling ramp k_{Ramp}

Figure 10.5: Sparse SMC4 diagnostics taken from printed status lines in the pinned hold run. These plots help explain how the firmware protections discussed in Chapter 8 behave on the real rig.

10.4 Commanded reference tracking / nudge mode (N1)

Objective. Demonstrate balance-with-motion behavior by commanding a sequence of base targets $\theta_{\text{ref}}(t)$ while maintaining upright balance. In the pinned dataset, only the linear controller is evaluated in nudge mode.

Key takeaways.

- The pinned run follows the commanded sequence and keeps the pendulum upright during the full motion.
- The reversal steps show finite rise times of about 3.1 s to 3.3 s and visible overshoot.
- No commanded step meets the fixed $\pm 1^\circ$ settling criterion within the available post-step window, so the nudge result should be read as a successful motion demonstration rather than a tightly settled servo result.

Step metrics. *Note:* A settle time of "–" means the fixed $\pm 1^\circ$ band was not reached within the available post-step window.

Table 10.3: Nudge/reference tracking step metrics (linear controller).

Step	Target (°)	Rise (s)	Overshoot (°)	Settle (s)	Steady-state error (°)	max $\lvert\alpha\rvert$ (°)
0	20.0	–	0.0	–	-2.43	3.31
1	-20.0	3.284	9.61	–	1.64	2.84
2	0.0	3.126	5.62	–	1.93	3.53

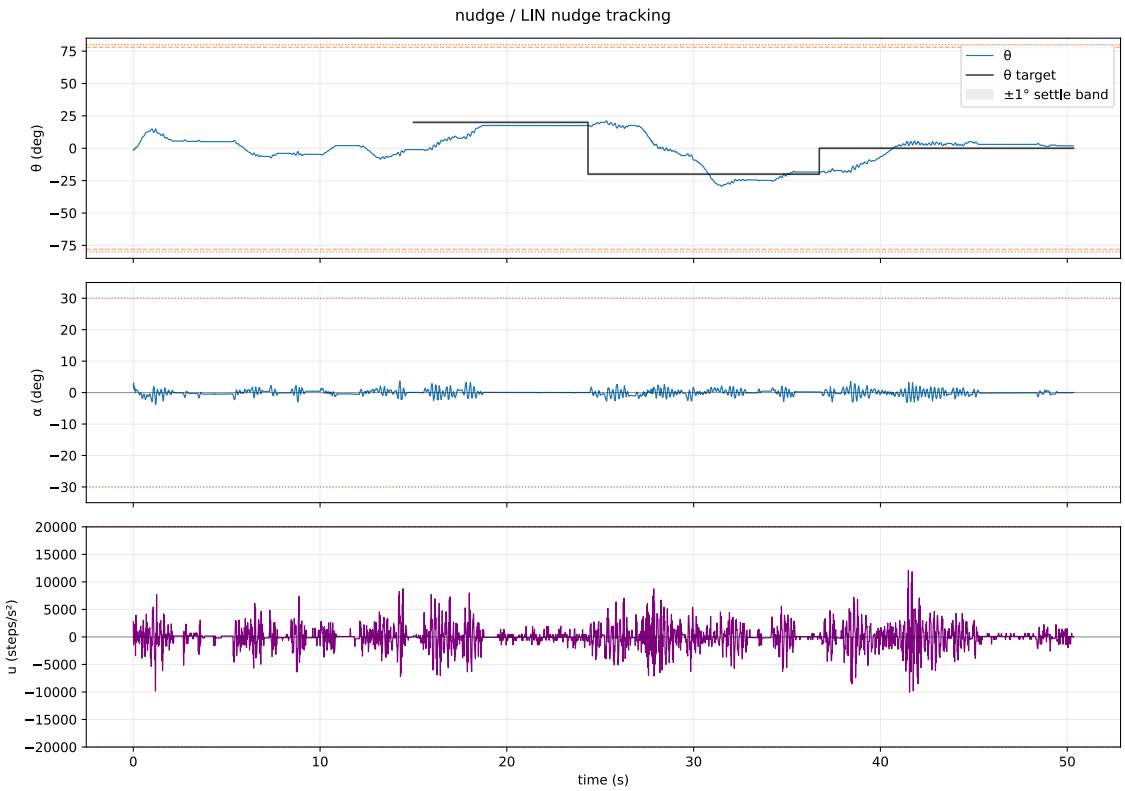


Figure 10.6: Pinned linear nudge run. The target sequence, measured base angle, visible ±1° settle band, pendulum angle, and actuation effort are shown together to make the missed settling intervals easy to read.

10.5 Finger tap disturbance (T1–T3)

Objective. While balancing upright, the pendulum is disturbed by brief manual taps. The goal is to recover without falling or hitting base limits. These runs do not contain explicit disturbance markers, so the comparison here is qualitative.

Key takeaways.

- The linear controller remains upright for the full pinned tap run.
- The hybrid SMC and full-surface SMC runs both end with a fallen outcome in this fixed dataset.
- In these representative runs, the hybrid SMC shows the largest peak angle excursion and the largest peak acceleration command, while the full-surface SMC shows the largest RMS acceleration effort.

Summary metrics.

Table 10.4: Summary metrics by controller mode (tap experiments).

Mode	Duration (s)	α RMS (°)	max $\lvert\alpha\rvert$ (°)	max $\lvert\theta\rvert$ (°)	θ drift (° s^{-1})	acc RMS (step s^{-2})	max $\lvert acc\rvert$ (step s^{-2})
Linear	31.293	0.864	5.92	37.7	0.0	1695.658	12717.0
Hybrid SMC	27.877	1.847	24.87	54.93	-0.155	1493.067	15384.0
Full-surface SMC	31.466	2.524	22.61	38.23	0.0	3843.042	12000.0

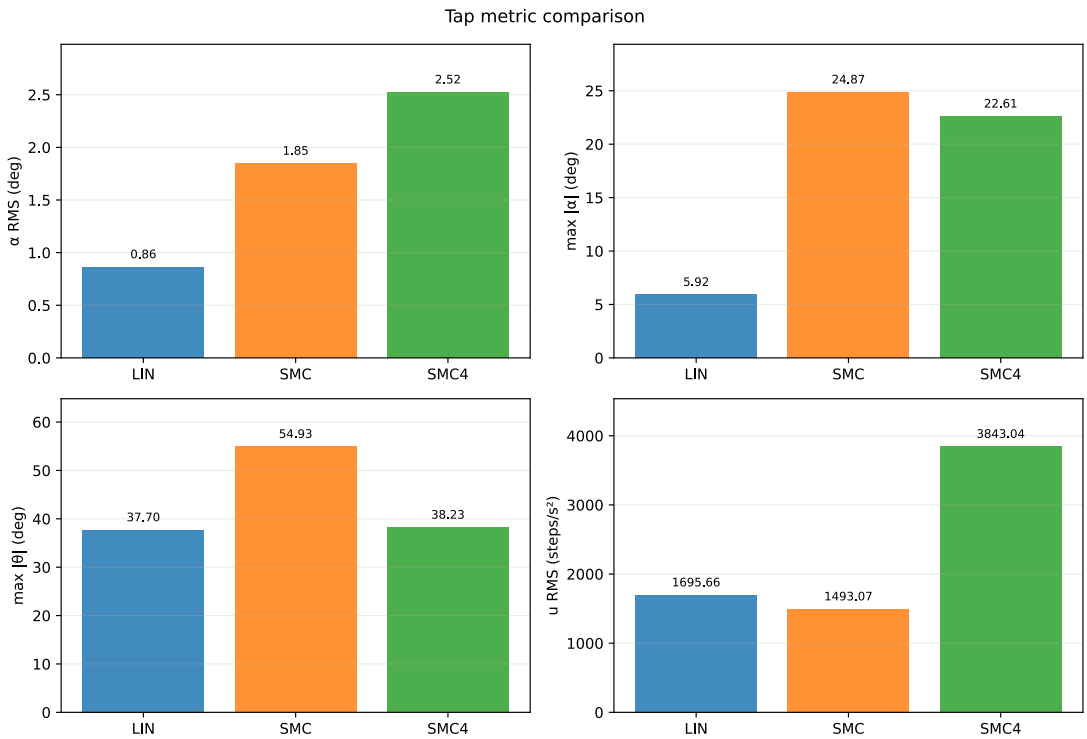


Figure 10.7: Compact comparison of the pinned tap metrics used in the discussion: α RMS, max $|\alpha|$, max $|\theta|$, and acceleration RMS.

Interpretation. The tap runs show the practical limit of this stepper-driven platform more clearly than the hold runs. When the controller asks for aggressive recovery, the measured state can say "recover harder" even when the open-loop stepper is already close to what it can realize. On this rig, that mismatch can keep the recovery in a high-effort region until the pendulum leaves the upright window or the base approaches a limit.

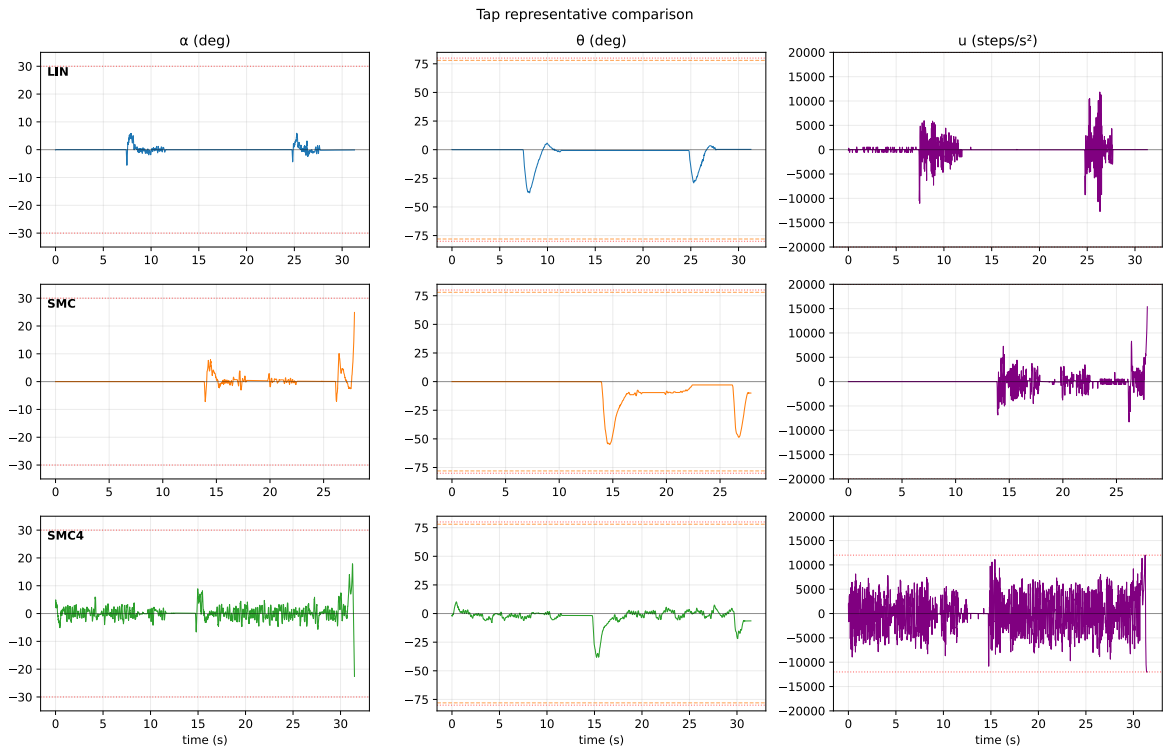
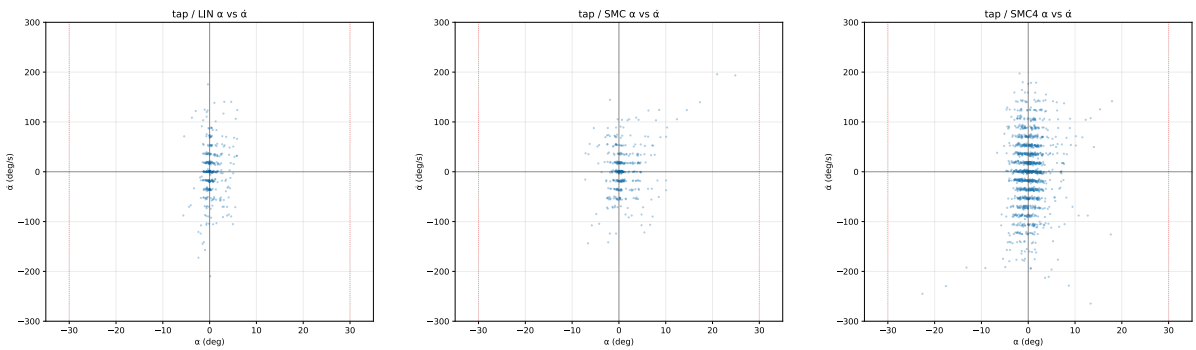


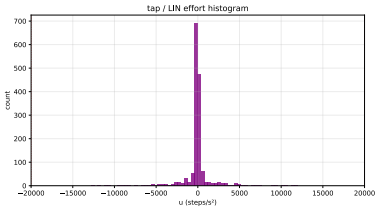
Figure 10.8: Common-scale comparison panel for the pinned tap runs. Shared scales make the recovery attempts and the effort differences easier to compare directly.

Supporting diagnostics

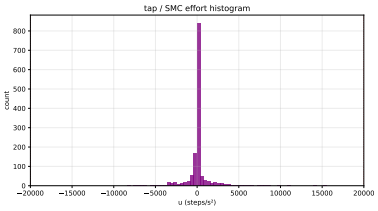


(a) Linear (b) Hybrid SMC (c) Full-surface SMC

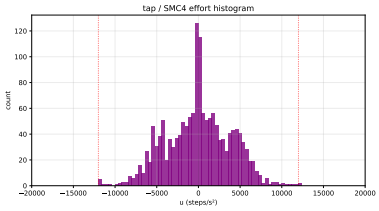
Figure 10.9: Phase portraits for the pinned tap runs. These show how much wider the disturbed state excursions are than in the hold case.



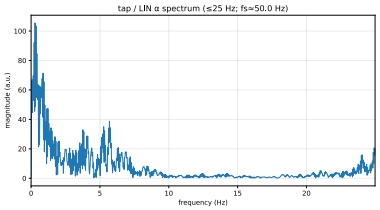
(a) Linear effort



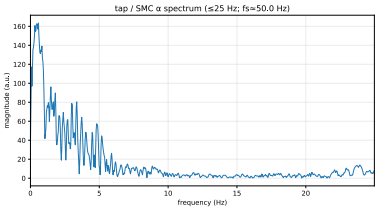
(b) Hybrid SMC effort



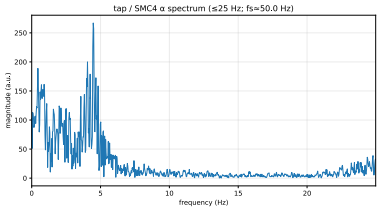
(c) Full-surface SMC effort



(d) Linear spectrum

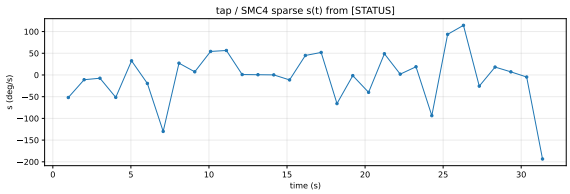


(e) Hybrid SMC spectrum

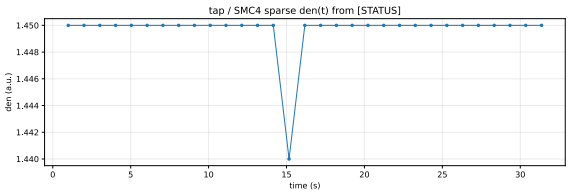


(f) Full-surface SMC spectrum

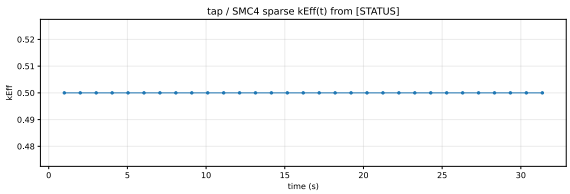
Figure 10.10: Top row: effort histograms for the pinned tap runs. Bottom row: low-frequency α spectra. The spectra are low-frequency only (≤ 25 Hz) because the thesis logs are recorded at about 50 Hz.



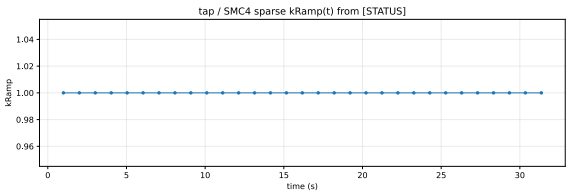
(a) Sliding surface $s(t)$



(b) Protected denominator



(c) Effective coupling k_{Eff}



(d) Coupling ramp k_{Ramp}

Figure 10.11: Sparse SMC4 diagnostics taken from printed status lines in the pinned tap run. These show how the denominator protection and ramping remain active during the disturbed recovery attempt.

10.6 Overall discussion and recommendations

Based on this rig and this fixed dataset:

- For calm steady hold-at-center behavior with low effort, the linear controller is the most consistent choice.
- For better peak suppression in the pinned hold run, the hybrid SMC is the strongest of the three, but its tap run shows that aggressive recovery can still fail on this platform.
- The full-surface SMC provides one unified nonlinear structure, but on this stepper-driven rig it is the most sensitive to actuator limits and protection logic.
- Nudge mode shows that commanded base motion while balancing is possible with the linear controller, but the pinned run does not show tight settling inside the fixed $\pm 1^\circ$ band.

Because each condition is represented by one pinned run only, these conclusions should be read as qualitative observations, not as statistical claims.

Chapter 11

Conclusion and Future Work

11.1 Conclusion

This project developed an upright-only control stack for a custom stepper-driven rotary (Furuta) inverted pendulum:

- A first-principles model was derived and specialized for the physical rig (Chapter 3).
- A practical firmware pipeline was implemented, including derivative filtering, wrap-safe angle handling, glitch rejection, engage logic, soft limits, stall diagnostics, and controlled disarm behavior (Chapters 4–5).
- Three upright controllers were implemented on the same hardware: linear full-state feedback, hybrid sliding mode control, and full-surface sliding mode control (Chapters 6–8).
- Commanded base-position moves while balancing were demonstrated with the linear controller through nudge mode (Chapter 10).

Across the pinned seven-run dataset, the linear controller was the most consistent overall choice on this rig. It gave the calmest hold behavior and remained upright in the pinned tap run. The hybrid SMC reduced peak upright excursions well in the pinned hold run, but its disturbed recovery was less reliable in the pinned tap run. The full-surface SMC also balanced upright in undisturbed conditions, but it was the most sensitive to actuator limits and protective shaping on this stepper-driven platform.

The nudge experiment showed that commanded base motion while balancing is practical on the real rig, but the pinned run should be read carefully: it demonstrates successful motion without a fall, not tight settling inside the fixed $\pm 1^\circ$ band.

These conclusions are qualitative because each condition is represented by one pinned run only. Stronger claims would require repeated trials for each controller and experiment type.

11.2 Future work

The main next steps suggested by this thesis are:

- **Repeat trials for stronger comparisons.** Run many trials per controller and per experiment type so future reports can show distributions instead of single representative runs.
- **Add explicit tap markers.** A button, trigger, or external sensor would make disturbance onset measurable and would allow cleaner recovery-time metrics.

- **Increase logging bandwidth.** Logging selected signals closer to the 200 Hz control rate would improve effort and timing analysis.
- **Use more actuator-aware nonlinear control.** The tap results show that actuator limits matter strongly on this platform. Future nonlinear designs should account for bounded actuation more directly.
- **Improve state estimation.** Better derivative estimation or observer-based methods could reduce noise sensitivity and false aborts.

Bibliography

[1] K. Furuta, M. Yamakita, and S. Kobayashi, "Swing-up control of inverted pendulum using pseudo-state feedback," in *Proceedings of IECON '91 (IEEE International Conference on Industrial Electronics)*, pp. 2193–2198, 1991.

[2] M. F. Hamza, H. J. Yap, I. A. Choudhury, A. I. Isa, A. Y. Zimit, and T. Kumbasar, "Current development on using rotary inverted pendulum as a benchmark for testing linear and nonlinear control algorithms," *Mechanical Systems and Signal Processing*, vol. 116, pp. 347–369, 2019.

[3] D. Ju, J. Lee, and Y. S. Lee, "An educational platform for real-time control and reinforcement learning experiments using rotary inverted pendulum and lw-rcp," in *Proc. 22nd Int. Conf. on Informatics in Control, Automation and Robotics (ICINCO)*, pp. 274–281, SciTePress, 2025.

[4] B. S. Cazzolato and Z. Prime, "On the dynamics of the furuta pendulum," *Journal of Control Science and Engineering*, vol. 2011, pp. 1–9, 2011.

[5] O. García-Alarcón, S. Puga-Guzmán, and J. Moreno-Valenzuela, "On parameter identification of the furuta pendulum," in *Procedia Engineering (Proc. of the IEEE/ASME Int. Conf. on Advanced Intelligent Mechatronics)*, vol. 35, pp. 77–84, 2012.

[6] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers.* Princeton, NJ: Princeton University Press, 2008.

[7] STMicroelectronics, "Introduction to the integrated rotary inverted pendulum system," tech. rep., STMicroelectronics, 2019. Application Note, STEVAL-EDUKIT01 (Motor Control Curriculum).

[8] M. Park, Y.-J. Kim, and J.-J. Lee, "Swing-up and lqr stabilization of a rotary inverted pendulum," in *Proc. 16th Int. Symp. on Artificial Life and Robotics (AROB)*, pp. 561–564, 2011.

[9] I. Chawla and A. Singla, "Real-time control of a rotary inverted pendulum using robust lqr-based anfis controller," *International Journal of Nonlinear Sciences and Numerical Simulation*, vol. 19, no. 5, pp. 379–389, 2018.

[10] K. Ogata, *Modern Control Engineering.* Prentice Hall, 5 ed., 2010.

[11] İ. Yiğit, "Model free sliding mode stabilizing control of a real rotary inverted pendulum," *Journal of Vibration and Control*, vol. 23, no. 10, pp. 1645–1662, 2017.

[12] N. P. Nguyen *et al.*, "Fuzzy-based super-twisting sliding mode stabilization control for under-actuated rotary inverted pendulum systems," *IEEE Access*, vol. 8, pp. 185079–185092, 2020.

[13] D.-B. Pham, Q.-T. Dao, and T.-V.-A. Nguyen, "Optimized hierarchical sliding mode control for the swing-up and stabilization of a rotary inverted pendulum," *Automation*, vol. 5, no. 3, pp. 282–296, 2024.

[14] V. I. Utkin, *Sliding Modes in Control and Optimization.* Berlin, Heidelberg: Springer, 1992.

[15] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control.* Prentice Hall, 1991.

[16] K. D. Young, V. I. Utkin, and Ümit Özguner, "A control engineer's guide to sliding mode control," *IEEE Control Systems Magazine*, vol. 19, no. 3, pp. 36–50, 1999.